

ros2行为树

一、核心概念与节点基础

1.节点类型

节点类型	英文名	子节点数量	说明
控制节点	ControNode	1.....N	通常根据其兄弟节点的结果或/和自身状态来tick一个子节点
装饰节点	DecorateNode	1	除其他功能外，它可以改变其子节点的结果，或多次tick其子节点
条件节点	ConditionNode	0	不应改变系统状态，不得返回“运行中”
动作节点	ActionNode	0	这是“执行操作”的节点

在这里，我们可以区分同步节点和异步节点

同步节点：

当一个任务流执行到该节点时，发起者必须等待该节点的任务完全结束，并得到一个明确的结果（成功或失败）后，才能继续向下执行

异步节点：

当一个任务流执行到这个节点，发起者不会等待该节点任务的完成，他只是“触发”一个任务，然后立即继续执行后面的操作，该节点的任务在后台独立运行，完成后通过回调等方式将结果告知发起者

2.第一种控制节点

最基本的控制节点：

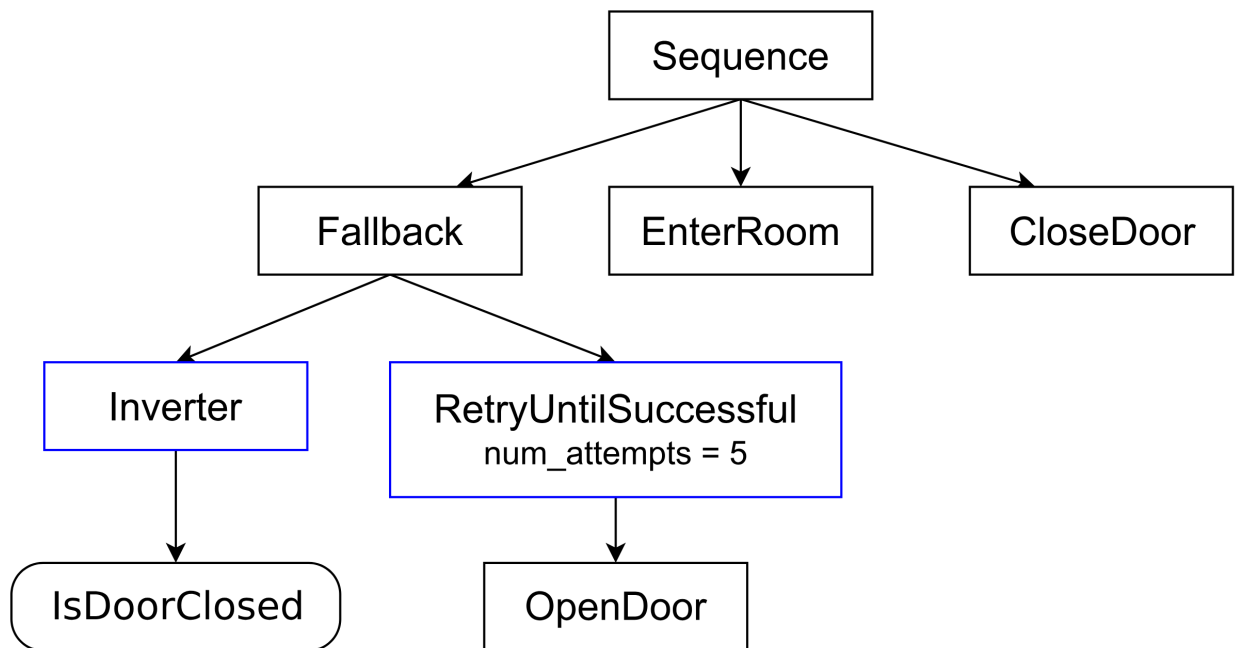
序列节点的子节点总是有序的，在图形中，执行顺序是从左到右

装饰节点：

根据装饰节点类型的不同，该节点的目标可以是以下之一

- 转换从子节点接收到的结果
- 停止子节点的执行

根据装饰器的类型，重复tick子节点



3.节点与树

必须创建自己的动作节点和条件节点，我们在部署行为树的时候，是基于xml的脚本语言来创建和组合

tick回调函数

任何树节点都可以看作是一种回调函数的机制，如下代码

代码块

```
1  // 可以封装到行为树动作中的最简单的回调函数
2  NodeStatus HelloTick()
3  {
4      std::cout << "Hello World\n";
5      return NodeStatus::SUCCESS;
6  }
7
8  // 允许库创建调用 HelloTick() 的动作节点
9  // (在教程中解释)
10 factory.registerSimpleAction("Hello", std::bind(HelloTick));
```

在上述代码中，我们通过函数指针（依赖注入）的方式，创建了一种调用HelloTick的特定类型的树节点，在定义自定义的树节点时，应该从TreeNode类继承，或者和继承其派生类：

ActionNodeBase

ConditionNode

二、创建第一个行为树

行为树是一种在正确时机，正确条件调用回调函数的机制，这些回调函数内部做什么完全取决于自己

1.创建树节点默认方式是通过继承

代码块

```

1  // 自定义同步动作节点（无端口）的示例
2  class ApproachObject : public BT::SyncActionNode
3  {
4  public:
5      ApproachObject(const std::string& name) :
6          BT::SyncActionNode(name, {})
7      {}
8
9      // 必须重写虚函数 tick()
10     BT::NodeStatus tick() override
11     {
12         std::cout << "ApproachObject: " << this->name() << std::endl;
13         return BT::NodeStatus::SUCCESS;
14     }
15 };

```

创建一个类（类的名字相当于xml中所定义的标签名），继承至BT库中的SyncActionNode,即同步动作节点，

在构造函数中，接收一个名字（比如在xml中对name属性的定义），然后传给Sync ActionNode。{}表示这个节点没有输入/输出端口

tick()函数是节点的核心。当行为树执行到这个节点时，就会调用。它必须始终返回一个NodeStatus，即running，success或failure，也可以使用依赖注入，通过一个函数指针来创建树节点

2.通过函数指针包装节点

可以把普通函数或类成员函数包装成节点

在我们创建的时候，函数的格式必须严格遵守下面的样式

代码块

```
1 BT::NodeStatus myFunction(BT::TreeNode& self)
```

- 返回类型必须是BT::NodeStatus
- 参数列表必须只能有一个参数：BT::TreeNode& self,它是一个节点的引用，代表当前正在执行的这个节点本身

普通函数示例

代码块

```
1 BT::NodeStatus CheckBattery() {  
2     std::cout << "[ Battery: OK ]" << std::endl;  
3     return BT::NodeStatus::SUCCESS;  
4 }
```

类成员函数示例

代码块

```
1 using namespace BT;  
2  
3 //我们想把open()和close()方法包装进动作节点  
4 class GripperInterface  
5 {  
6 public:  
7     GripperInterface():_open(true){}  
8  
9     NodeStatus open()  
10    {  
11        _open=true;  
12        std::cout<<"GripperInterface::open"<<std::endl;  
13        return NodeStatus::SUCCESS;  
14    }  
15  
16    NodeStatus close()  
17    {  
18        std::cout<<"GripperInterface::close"<<std::endl;  
19        open_=false;  
20        return NodeStatus::SUCCESS;  
21    }  
22  
23 private:  
24     bool _open;//共享信息  
25 };
```

3.用XML定义树

代码块

```
1  <root BTCPP_format="4">
2    <BehaviorTree ID="MainTree">
3      <Sequence name="root_sequence">
4        <CheckBattery name="check_battery"/>
5        <OpenGripper name="open_gripper"/>
6        <ApproachObject name="approach_object"/>
7        <CloseGripper name="close_gripper"/>
8      </Sequence>
9    </BehaviorTree>
10 </root>
```

必须先将自定义的树节点注册到BehaviorTreeFactory中，然后从文件或文本加载XML。

标签名（如：CheckBattery）必须和注册时用的ID一致

name属性是实例名称，方便调试，可重复

4.在C++中注册节点并加载树

代码块

```
1  #include "behaviortree_cpp/bt_factory.h"
2
3  // 包含自定义节点定义的文件
4  #include "dummy_nodes.h"
5  using namespace DummyNodes;
6
7  int main()
8  {
9      // 使用 BehaviorTreeFactory 注册我们的自定义节点
10     BehaviorTreeFactory factory;
11
12     // 创建节点的推荐方式是通过继承
13     factory.registerNodeType<ApproachObject>("ApproachObject");
14
15     // 使用函数指针注册 SimpleActionNode
16     // 可以使用 C++11 lambda 或 std::bind
17     factory.registerSimpleCondition("CheckBattery", [&](TreeNode&) {
18         return CheckBattery(); });
19
20     // 也可以使用类的方法创建 SimpleActionNode
21     GripperInterface gripper;
```

```

21     factory.registerSimpleAction("OpenGripper", [&](TreeNode&){ return
gripper.open(); } );
22     factory.registerSimpleAction("CloseGripper", [&](TreeNode&){ return
gripper.close(); } );
23
24     // 树是在部署时创建的（即运行时，但只在开始时创建一次）
25
26     // 重要：当 "tree" 对象超出作用域时，所有树节点都会被销毁
27     auto tree = factory.createTreeFromFile("./my_tree.xml");
28
29     // 要“执行”一棵树，你需要对它“滴答”
30     // 滴答信号根据树的逻辑传播给子节点
31     // 在此例中，因为序列的所有子节点都返回 SUCCESS，所以整个序列被执行
32     tree.tickWhileRunning();
33
34     return 0;
35 }
36
37 /* 预期输出：
38 *
39 [ Battery: OK ]
40 GripperInterface::open
41 ApproachObject: approach_object
42 GripperInterface::close
43 */

```

三、数据共享：黑板与端口

1. 黑板的概念

在BehaviorTree.CPP中，黑板是一个键值对存储，用于在行为树的不同节点之间共享数据。它就像一对公共的“黑板”，任何人都可以在上面写下信息（写数据），也可以读取其他系统但写下的信息（读数据），从而实现结点间的数据交换，而不需要节点之间直接耦合

2. 黑板的作用

- 数据共享：让树中的不同节点（比如条件节点、动作节点）能够访问共同的数据，例如机器人的目标位置、传感器读数、状态标志等
- 解耦节点：节点不再需要知道数据来自哪个具体的系统，只需要知道黑板上的某个键（key）即可，这使得节点可以被复用，只要黑板上有它需要的数据
- 支持端口映射：节点通过端口（port）声明自己需要哪些输入数据、会产生哪些输出数据。在XML中，你可以将这些端口“重映射”到黑板的某个键上，从而把节点的数据和黑板连接起来

3.黑板在XML中的使用

在XML中，黑板键用花括号{}表示，例如

代码块

```
1 <SaySomething message="{my_message}"/>
```

表示SaySomthing节点的message端口会从黑板上读取键名为my_message的值，如果黑板上有my_message，那么节点就会用这个值来输出

同样也可以把节点的输出写入黑板

代码块

```
1 <StoreValue value="{result}"/>
```

4.端口的概念

端口是声明自己需要哪些输入、会产生哪些输出的接口

在XML中，端口的赋值有两种方式

- 静态字符串：直接写值，例如message="hello world"
- 黑板键引用：用大括号括起来，例如message="{greetings}"，表示从黑板的greetings读取键值（或写入键值

代码块

```
1 <SaySomething name="first" message="hello world" />
2 <SaySomething name="second" message="{greetings}" />
```

第一个节点中，端口接收到字符串"hello world"

第二个节点中，被要求使用条目"greetings"在黑板上查找该值

5.定义一个带端口的节点

以动作节点SaySomething为例

代码块

```
1 // 带输入端口的同步动作节点
2 class SaySomething : public SyncActionNode
3 {
4     public:
```

```

5 // 如果你的节点有端口，你必须使用这个构造函数签名
6 SaySomething(const std::string& name, const NodeConfig& config)
7     : SyncActionNode(name, config)
8 { }
9
10 // 必须定义这个静态方法
11 static PortsList providedPorts()
12 {
13     // 这个动作有一个名为 "message" 的输入端口
14     return { InputPort<std::string>("message") };
15 }
16
17 // 重写虚函数 tick()
18 NodeStatus tick() override
19 {
20     Expected<std::string> msg = getInput<std::string>("message");
21     // 检查期望值是否有效。如果无效，抛出错误
22     if (!msg)
23     {
24         throw BT::RuntimeError("缺少必需的输入 [message]: ",
25                                 msg.error() );
26     }
27     // 使用 value() 方法提取有效的消息
28     std::cout << "机器人说: " << msg.value() << std::endl;
29     return NodeStatus::SUCCESS;
30 }
31 };

```

- providedPorts()告诉工厂这个节点有一个输入端口，名为message，类型是std::string
- 在tick()中用getInput<T>("端口名")读取数据（返回一个Expected<T>对象，需要检查有效性）
- 输出端口类似，用setOutput("端口名", 值)写入黑板值

6.输出端口

这里是用ThinkWhatToSay使用输出端口将字符串写入条目的节点示例

代码块

```

1 class ThinkWhatToSay : public SyncActionNode
2 {
3 public:
4     ThinkWhatToSay(const std::string& name, const NodeConfig& config)
5         : SyncActionNode(name, config)
6     { }

```



```

7
8     static PortsList providedPorts()
9     {
10         return { OutputPort<std::string>("text") };
11     }
12
13     // 这个动作将一个值写入端口 "text"
14     NodeStatus tick() override
15     {
16         // 输出可能在每次 tick() 时改变。这里我们保持简单。
17         setOutput("text", "答案是42");
18         return NodeStatus::SUCCESS;
19     }
20 };

```

为了调试，可以使用名为Script的内置动作将一个静态值写入条目

代码块

```

1   <Script code=" the_answer:='答案是42' " />

```

7. 输入端口和输出端口连接

通过相同的黑板键实现连接

代码块

```

1   <Sequence>
2       <SaySomething message="hello" />           <!-- 静态输入 -->
3       <ThinkWhatToSay text="{the_answer}" />     <!-- 输出到黑板键
the_answer -->
4       <SaySomething message="{the_answer}" />     <!-- 从黑板键
the_answer 输入 -->
5   </Sequence>

```

- ThinkWhatToSay有一个输出端口text，它把值写入黑板的the_answer键。
- 第二个SaySomething从同一个键the_answer读取数据，从而获得前一个节点的输出

8. 示例

这个示例中，执行了一个由三个动作组成的序列

- 动作1从静态字符串读取输入信息
- 动作2将一些内容写入黑板中名为the_answer的条目

- 动作3从黑板中名为the_answer的条目读取输入消息

代码块

```
1  <root BTCPP_format="4" >
2      <BehaviorTree ID="MainTree">
3          <Sequence name="root_sequence">
4              <SaySomething      message="hello" />
5              <ThinkWhatToSay    text="{the_answer}"/>
6              <SaySomething      message="{the_answer}" />
7          </Sequence>
8      </BehaviorTree>
9  </root>
```

然后我们再注册并执行树的C++代码

代码块

```
1  #include "behaviortree_cpp/bt_factory.h"
2
3  // 包含自定义节点定义的文件
4  #include "dummy_nodes.h"
5  using namespace DummyNodes;
6
7  int main()
8  {
9      BehaviorTreeFactory factory;
10     factory.registerNodeType<SaySomething>("SaySomething");
11     factory.registerNodeType<ThinkWhatToSay>("ThinkWhatToSay");
12
13     auto tree = factory.createTreeFromFile("./my_tree.xml");
14     tree.tickWhileRunning();
15     return 0;
16 }
17
18 /* 预期输出:
19     机器人说: hello
20     机器人说: 答案是42
21 */
```

我们使用相同的键（the_answer）将输出端口“连接”到输入端口；换句话说，它们“指向”黑板的同一个条目

- 第一个SaySomething直接输出静态字符串“hello”
- ThinkWhatToSay将“The answer is 42”写入黑板的the_answer键

- 第二个SaySomething从黑板读取the_answer并输出

四、进阶数据流：泛型端口

1. 泛型端口的作用

在实际机器人应用中，节点之间穿的数据往往不是简单的字符串，可能是坐标、速度、姿态等自定义结构体，泛型端口允许直接在节点间传递这些类型，同时保持XML的灵活性

2. 核心机制：字符串 ↔ 自定义类型的转换

由于XML中所有属性值本质都是字符串，当端口类型不是std::string时，库需要知道如何将字符串解析成你的自定义类型，这个转换是通过特化BT::convertFromString模板函数实现的
例如：

代码块

```
1 // 我们想使用这个自定义类型
2 struct Position2D
3 {
4     double x;
5     double y;
6 };
```

为了允许XML加早期从字符串实例化Position2D，我们需要提供一个BT::convertFromString<Position2D>(StringView)的模板特化

代码块

```
1 // 模板特化，用于将字符串转换为 Position2D。
2 namespace BT
3 {
4     template <> inline Position2D convertFromString(StringView str)
5     {
6         // 我们期望由分号分隔的实数
7         auto parts = splitString(str, ';');
8         if (parts.size() != 2)
9         {
10             throw RuntimeError("无效输入");
11         }
12         else
13         {
14             Position2D output;
15             output.x = convertFromString<double>(parts[0]);
16             output.y = convertFromString<double>(parts[1]);
```

```

17         return output;
18     }
19 }
20 } // end namespace BT

```

- `StringView`是轻量级字符串，可接受`std::string`或`const char*`
- `splitString`是库提供的辅助函数，也可以使用其它字符串分割方法
- `convertFromString<double>`是库你内置的`double`转换，这里复用它来转换每个坐标分量
- 这样，当XML中出现"-1;3"这样的字符串并赋值给`Position2D`端口时，库会自动调用这个特化函数完成转换

3.示例1

我们可以创建两个自定义动作，一个会写入端口，另一个会从端口读取

代码块

```

1  class CalculateGoal: public SyncActionNode
2  {
3      public:
4          CalculateGoal(const std::string& name, const NodeConfig& config):
5              SyncActionNode(name,config)
6          {}
7
8          static PortsList providedPorts()
9          {
10             return { OutputPort<Position2D>("goal") };
11          }
12
13          NodeStatus tick() override
14          {
15              Position2D mygoal = {1.1, 2.3};
16              setOutput<Position2D>("goal", mygoal);
17              return NodeStatus::SUCCESS;
18          }
19  };
20
21  class PrintTarget: public SyncActionNode
22  {
23      public:
24          PrintTarget(const std::string& name, const NodeConfig& config):
25              SyncActionNode(name,config)
26          {}
27
28          static PortsList providedPorts()

```

```

29     {
30         // 可选地，端口可以有一个人类可读的描述
31         const char* description = "在控制台上简单地打印目标...";
32         return { InputPort<Position2D>("target", description) };
33     }
34
35     NodeStatus tick() override
36     {
37         auto res = getInput<Position2D>("target");
38         if( !res )
39         {
40             throw RuntimeError("读取端口 [target] 时出错:", res.error());
41         }
42         Position2D target = res.value();
43         printf("目标位置: [ %.1f, %.1f ]\n", target.x, target.y );
44         return NodeStatus::SUCCESS;
45     }
46 };

```

我们把这个程序分成两个部分

第一部分:CalculateGoal类（输出节点）

代码块

```

1  class CalculateGoal: public SyncActionNode {
2  public:
3      CalculateGoal(const std::string& name, const NodeConfig& config):
4          SyncActionNode(name, config) {}
5
6      static PortsList providedPorts() {
7          return { OutputPort<Position2D>("goal") };
8      }
9
10     NodeStatus tick() override {
11         Position2D mygoal = {1.1, 2.3};
12         setOutput<Position2D>("goal", mygoal);
13         return NodeStatus::SUCCESS;
14     }
15 };

```

作用：

- 计算一个目标位置（这里是硬编码的(1.1,2.3)，并将他写入黑板，供其他节点使用
- 这是一个输出节点：通过输出端口将数据“发布”到黑板

关键：

- 构造函数：必须接受name和config，并传递给基类SyncActionNode，因为带端口的节点需要NodeConfig来获取黑板引用等信息
- providePorts()：静态方法，声明拥有的端口，这里声明了一个名为“goal”的输出端口，类型为Position2D
- tick()
 - 创建一个Position2D对象mygoal
 - 调用setOutput<Position2D>("goal",mygoal)将mygoal的值写入与该输出端口关联的黑板条目（具体的黑板键由XML中的重映射决定）
 - 返回SUCCESS表示动作立即完成

第二部分：PrintTarget类（输入节点）

代码块

```
1  class PrintTarget: public SyncActionNode {
2  public:
3      PrintTarget(const std::string& name, const NodeConfig& config):
4          SyncActionNode(name, config) {}
5
6      static PortsList providedPorts() {
7          const char* description = "在控制台上简单地打印目标...";
8          return { InputPort<Position2D>("target", description) };
9      }
10
11     NodeStatus tick() override {
12         auto res = getInput<Position2D>("target");
13         if( !res ) {
14             throw RuntimeError("读取端口 [target] 时出错:", res.error());
15         }
16         Position2D target = res.value();
17         printf("目标位置: [ %.1f, %.1f ]\n", target.x, target.y);
18         return NodeStatus::SUCCESS;
19     }
20 };
```

作用：

- 从黑板读取一个目标位置，并打印到控制台
- 它是一个输入节点：通过输入端口从黑板获取数据

关键：

- 构造函数：同样需要name和config
- providedPorts(): 声明一个名为“target”的输入端口，类型为Position2D，并附带一个描述字符串（可选）
- tick():
 - 调用getInput<Position2D>("target")从黑板取值，返回值是一个Expected<Position2D>对象，它封装了可能存在的错误
 - 检查res是否有效：如果无效（例如端口未赋值、类型不匹配或黑板键不存在），则抛出运行时异常
 - 如果有效，通过res.value()获取实际的Position2D对象，并打印其坐标
 - 返回SUCCESS

4.示例2

这个示例中的树是一个由4个动作组成的序列

- 使用CalculateGoal动作，在GoalPosition条目中存储一个Position2D值
- 调用PrintTarget。其输入“target”将从黑板条目GoalPosition中读取
- 使用内置动作Script将字符串“-1;3”赋值给键OtherGoal。从字符串到Position2D的转换将自动完成
- 再次调用PrintTarget。其输入“target”将从条目OtherGoal中读取

代码块

```

1  static const char* xml_text = R"(
2
3  <root BTCPP_format="4" >
4      <BehaviorTree ID="MainTree">
5          <Sequence name="root">
6              <CalculateGoal goal="{GoalPosition}" />
7              <PrintTarget   target="{GoalPosition}" />
8              <Script        code=" OtherGoal:='-1;3' " />
9              <PrintTarget   target="{OtherGoal}" />
10         </Sequence>
11     </BehaviorTree>
12 </root>
13 );"
14
15 int main()
16 {
17     BT::BehaviorTreeFactory factory;
18     factory.registerNodeType<CalculateGoal>("CalculateGoal");
19     factory.registerNodeType<PrintTarget>("PrintTarget");

```

```

20
21     auto tree = factory.createTreeFromText(xml_text);
22     tree.tickWhileRunning();
23
24     return 0;
25 }
26 /* 预期输出:
27
28     目标位置: [ 1.1, 2.3 ]
29     转换字符串: "-1;3"
30     目标位置: [ -1.0, 3.0 ]
31 */

```

在其中，我们定义了一个XML行为树

代码块

```

1  <root BTCPP_format="4">
2    <BehaviorTree ID="MainTree">
3      <Sequence name="root">
4        <CalculateGoal goal="{GoalPosition}" />
5        <PrintTarget target="{GoalPosition}" />
6        <Script code=" OtherGoal:='-1;3' " />
7        <PrintTarget target="{OtherGoal}" />
8      </Sequence>
9    </BehaviorTree>
10 </root>

```

这是一个序列节点，包含了四个子节点，按顺序执行：

- <CalculateGoal goal="{GoalPosition}" />

调用自定义节点CalculateGoal，其输出端口goal被重映射到黑板键GoalPosition，节点执行时会将一个硬编码的Position2D值写入

- <PrintTarget target="{GoalPosition}" />

调用自定义节点PrintTarget，其输入端口target重映射到同一个黑板键GoalPosition，因此它会读取之前写入的值并打印

- <Script code=" OtherGoal:='-1;3' " />

内置的Script节点执行一段简单的脚本，将字符串“-1;3”赋值给黑板键OtherGoal。这里的语法：OtherGoal:='-1;3'表示将该字符串写入黑板键OtherGoal

- <PrintTarget target="{OtherGoal}" />

再次调用PrintTarget，但这次输入端口target重映射到黑板键OtherGoal。由于该键存储的是字符串“-1;3”，而PrintTarget的端口类型是Position2D，库会自动调用convertFromString将字符串解析为Position2D(-1.0,3.0)，然后打印

接下来，在C++代码中调用它

代码块

```
1  int main() {
2      BT::BehaviorTreeFactory factory;
3      factory.registerNodeType<CalculateGoal>("CalculateGoal");
4      factory.registerNodeType<PrintTarget>("PrintTarget");
5
6      auto tree = factory.createTreeFromText(xml_text);
7      tree.tickWhileRunning();
8
9      return 0;
10 }
```

- 使用BT库内置的工厂类，用于注册自定义节点类型
- 注册节点：将CalculateGoal和PrintTarget类分别于XML标签名“CalculateGoal”和“PrintTarget”关联
- 从文本创建树：createTreeFromText解析XML字符串，构建行为树实例。此时，工厂会检查端口类型是否匹配，并建立黑板于端口的连接
- 执行树：tickWhileRunning()反复tick根节点（这里是序列），直到树返回成功或失败。由于所有子节点都返回SUCCESS，序列回一次执行所有子节点一次，然后返回成功，tickWhileRunning停止

五、最佳实践：端口与黑板的关系

端口是一种为黑板添加了一层间接寻址和额外语义的接口

1.本质区别

- 黑板是一个全局的键值存储，任何节点都可以直接读写任意键
- 端口是在节点上定义的“接口”，通过它可以声明节点需要什么输入、产生什么输出，然后在XML中将这端口“重映射”到黑板具体键上

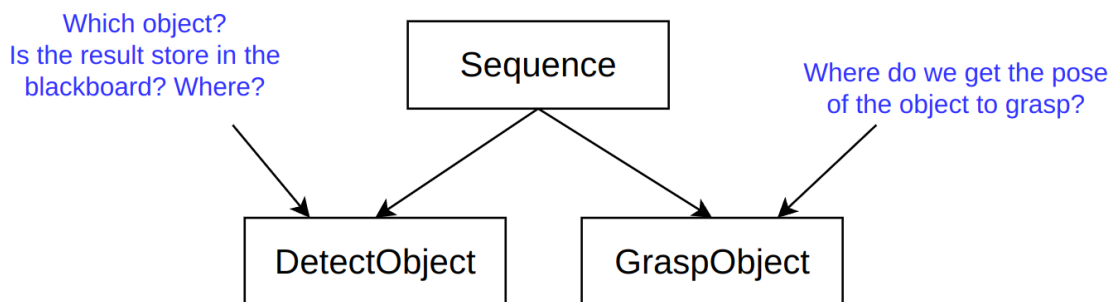
端口相当于在黑板上增加了一层间接访问和语义约束，让数据流更清晰、可维护

2.推荐使用端口的原因

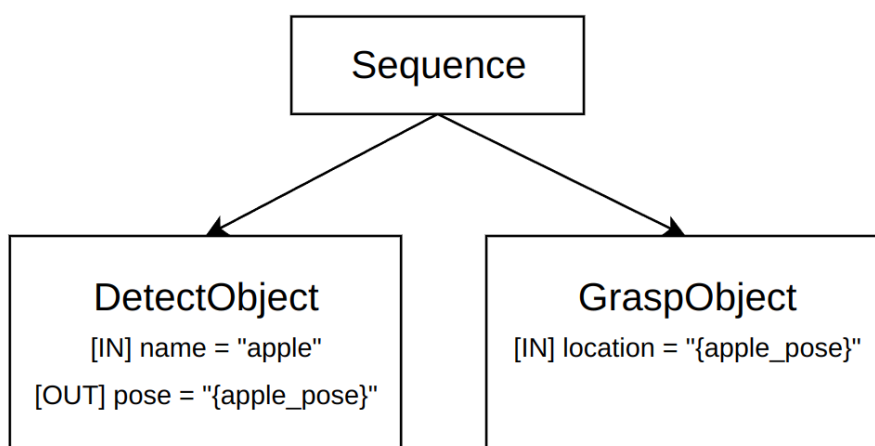
2.1支持模型驱动开发

节点之间的数据流的描述必须是模型的一部分。此外，还想明确表达黑板条目是被写入（输出）、读取（输入）还是两者兼具

无端口示意图：



如果输入/输出端口是模型的一部分，节点的意图及其与其他节点的关系就变得更加明确



端口是节点模型的一部分。可以通过查看节点的providedPorts(), 你可以知道它需要什么输入、会产生什么输出，而无需阅读代码或文档

外部工具如（Groot2）可以利用这些模型来提供可视化编辑，静态检查等功能

对比：如果节点直接读写黑板，只能通过阅读源码或文档来了解它的数据依赖，这容易出错且难以维护

2.2节点的可组合型与子树作用域

理想情况下，允许行为设计者构建由不同供应商/用户事项的树（即“组合节点”）

但是，当直接使用黑板时，名称冲突会立即成为一个问题

- 不同开发者编写的节点可能是用相同的黑板键名（如target），但含义和类型可能完全不同。如果直接是用黑板，这些节点就无法在同一个树中共存
- 端口通过重映射解决了这个问题：即使节点内部使用target端口，可以在XML上将它映射到任何黑板键（如move_target或grasp_target），从而避免冲突

2.3子树作用域

- 每个子树拥有独立的黑板实例，端口重映射让子树内部使用的键名与外部解耦，保证了子树的复用性和隔离性
- 如果节点直接读写黑板，则会破坏这种隔离，使得子树内部的黑板键与外部混合，难以复用

3.直接访问黑板的坏处

应该这样写：

代码块

```
1 // 在你的 tick() 中的示例代码
2 getInput("goal", goal);
3 setOutput("result", result);
```

不应该这样写：

代码块

```
1 // 不推荐：直接访问黑板
2 config().blackboard->get("goal", goal);
3 config().blackboard->set("result", result);
```

- 硬编码键名：“goal”和“result”固定在了代码中，要改变它们必须重新编译程序，而使用端口时，可以在XML中动态重映射
- 缺乏自描述：别人无法通过节点声明知道它使用了哪些黑板键，只能阅读源码或文档
- 工厂无法检查：BehaviorTreeFactory在加载XML时无法验证这些直接访问的键是否有效、类型是否正确，容易在运行时出错
- 与子树机制不兼容：直接访问黑板会绕过子树的隔离性，导致数据混乱
- 转换函数失效：如果你为自定义类型定义了convertFromString，它只会在端口赋值时自动调用；直接使用黑板则不会触发这个转换，导致类型解析困难

4.黑板的零拷贝访问（进阶）

在BehaviorTree.CPP中，默认情况下黑板使用值语义，即通过getInput和setOutput读写黑板条目时，会复制数据。这对于小型数据没有问题，但是对于大型或不可拷贝对象（如点云、图像等），复制开销很大，甚至不可能，此时需要引用语义——直接访问黑板中存储的对象本身

方法一：使用std::shared_ptr包装对象（值语义的“假引用”）

假设有这样一个可简单的行为树：

```

代码块<root BTCPP_format="4" >
2     <BehaviorTree ID="SegmentCup">
3         <Sequence>
4             <AcquirePointCloud ccloud="{pointcloud}"/>
5             <SegmentObject obj_name="cup" ccloud="{pointcloud}"
obj_pose="{pose}"/>
6         </Sequence>
7     </BehaviorTree>
8 </root>

```

AcquirePointCloud将写入黑板条目pointcloud

SegmentObject将从该条目读取

我们使用端口来进行访问：

代码块

```

1  PortsList AcquirePointCloud::providedPorts()
2  {
3      return { OutputPort<std::shared_ptr<Pointcloud>>("ccloud") };
4  }
5
6  PortsList SegmentObject::providedPorts()
7  {
8      return { InputPort<std::string>("obj_name"),
9              InputPort<std::shared_ptr<Pointcloud>>("ccloud"),
10             OutputPort<Pose3D>("obj_pose") };
11  }

```

- 如代码所示，在端口中声明类型为std::shared_ptr<T>，这样getInput/setOutput复制的是智能指针（开销小），但多个节点通过指针共享同一个堆上的T对象
- getInput和setOutput方法可以像往常一样使用，并且仍然具有值语义，但由于被复制的对象是shared_ptr,实际上是通过引用来访问Pointcloud实例的
- 缺点：不是线程安全，如果多个节点（尤其是异步节点）在不同线程中同时访问同一个裸指针指向的对象，可能产生数据竞争，因此不推荐在多线程环境使用

方法二：使用getLockedPortContent+castPtr（线程安全）

首先，在创建端口时，可以直接使用普通的Pointcloud，无需将其包装在std::shared_ptr中：

代码块

```

1  PortsList AcquirePointCloud::providedPorts()

```

```

2  {
3      return { OutputPort<Pointcloud>("cloud") };
4  }
5
6  PortsList SegmentObject::providedPorts()
7  {
8      return { InputPort<std::string>("obj_name"),
9              InputPort<Pointcloud>("cloud"),
10             OutputPort<Pose3D>("obj_pose") };
11 }

```

然后通过指针/引用访问Pointcloud的实例：

代码块

```

1  // 在下面的作用域内，只要 "any_locked" 存在，保护 "cloud" 实例的互斥锁就会保持锁定状态
2  if(auto any_locked = getLockedPortContent("cloud"))
3  {
4      if(any_locked->empty())
5      {
6          // 黑板中的条目尚未初始化。
7          // 你可以通过以下方式初始化它：
8          any_locked.assign(my_initial_pointcloud);
9      }
10     else if(Pointcloud* cloud_ptr = any_locked->castPtr<Pointcloud>())
11     {
12         // 成功转换为 Pointcloud* (原始类型)。
13         // 使用 cloud_ptr 修改 pointcloud 实例
14     }
15 }

```

- 端口声明时，使用原始类型（如Pointcloud），不包裹智能指针，在需要访问时，调用getLockedPortContent("port_name")获得一个LockedPortContent对象，它持有黑板条目的锁，然后通过castPtr<T>()获取类型安全的指针
- 如果条目尚未初始化，可以调用assign()赋值
- 必须在tick()中获取锁，并且尽快释放，（即让LockedPortContent对象析构），避免长时间锁住黑板

六、异步动作：原理与实现

1. 异步动作的概念

指那些不会立即完成，需要花费较长时间才能返回成功/失败的动作（例如导航，移动机械臂等）

一个异步动作需要满足以下要求

- 它不应该在tick()方法中阻塞太长时间，执行流程应尽快返回，不能使用sleep长时间等待
- 如果树的其他部分不再需要这个动作（例如条件失败），应该通过halt()方法立即停止它

2.实现异步动作——StatefulActionNode

StatefulActionNode是BehavioTree.CPP提供的辅助类，专门用于实现有状态的异步动作，它把tick()拆分成三个回调函数，基于StatefulActionNode的派生类必须重写一下虚方法，而不是tick()

方法	何时调用	作用
onStart()	节点第一次从空闲（IDLE）转为运行	初始化动作，发送请求等，可以立即返回SUCCESS/FALILURE，或者返回RUNNING表示开始执行
onRunning())	当节点处于RUNNING状态时，每次tick都会调用	检查进度、轮询结果。如果完成则返回SUCCESS/FAILURE，否则继续RUNNING
onHalted()	当节点被其它节点中断时调用（例如父节点不再需要它）	清理资源、取消请求等

示例

代码块

```
1  // 自定义类型
2  struct Pose2D
3  {
4      double x, y, theta;
5  };
6
7  namespace chr = std::chrono;
8
9  class MoveBaseAction : public BT::StatefulActionNode
10 {
11     public:
12         // 任何带有端口的树节点都必须使用此签名的构造函数
13         MoveBaseAction(const std::string& name, const BT::NodeConfig& config)
14             : StatefulActionNode(name, config)
15     {}
```

```

16
17 // 必须定义此静态方法
18 static BT::PortsList providedPorts()
19 {
20     return{ BT::InputPort<Pose2D>("goal") };
21 }
22
23 // 此函数在开始时被调用一次
24 BT::NodeStatus onStart() override;
25 //1. 读取输入目标
26 //2. 发送导航请求
27 //3. 设置预计完成时间 (这里模拟220ms后完成)
28 //4. 返回RUNNING
29
30 // 如果 onStart() 返回 RUNNING, 我们将持续调用
31 // 此方法, 直到它返回除 RUNNING 以外的内容
32 BT::NodeStatus onRunning() override;
33 //每次tick检查是否到达完成时间
34 //如果到了, 返回SUCCESS, 否则返回RUNNING
35
36 // 如果动作被另一个节点中止, 则执行此回调
37 void onHalted() override;
38
39 private:
40     Pose2D _goal;
41     chr::system_clock::time_point _completion_time;
42 };
43
44 //-----
45
46 BT::NodeStatus MoveBaseAction::onStart()
47 {
48     if ( !getInput<Pose2D>("goal", _goal))
49     {
50         throw BT::RuntimeError("缺少必需的输入 [goal]");
51     }
52     printf("[ MoveBase: 发送请求 ]. 目标: x=%f y=%f theta=%f\n",
53         _goal.x, _goal.y, _goal.theta);
54
55     // 我们使用此计数器模拟需要一定时间 (200 毫秒) 才能完成的操作
56     _completion_time = chr::system_clock::now() + chr::milliseconds(220);
57
58     return BT::NodeStatus::RUNNING;
59 }
60
61 BT::NodeStatus MoveBaseAction::onRunning()
62 {

```

```

63 // 假装我们正在检查是否已收到回复
64 // 你不希望在此函数内阻塞太长时间
65 std::this_thread::sleep_for(chr::milliseconds(10));
66
67 // 假装经过一定时间后，操作已完成
68 if(chr::system_clock::now() >= _completion_time)
69 {
70     std::cout << "[ MoveBase: 完成 ]" << std::endl;
71     return BT::NodeStatus::SUCCESS;
72 }
73 return BT::NodeStatus::RUNNING;
74 }
75
76 void MoveBaseAction::onHalted()
77 {
78     printf("[ MoveBase: 已中止 ]");
79 }

```

onRunning()中使用了sleep_for(10ms)来模拟轮询间隙，但真实的代码不应阻塞这么久，而应该检查非阻塞的信号或状态，这里sleep只是为了演示

在设计反应式行为树时，理解两个主要概念非常重要：

- 异步动作与同步动作的含义
- 在BT.CPP下，并发与并行之间的区别

3.并发vs并行

- 并发：多个任务可以在时间上重叠地开始、运行和完成，但不一定要同时运行，在单线程中，通过快速切换也能实现并发
- 多个任务真正同时运行，通常需要多核处理器和多线程

BT.CPP的树执行引擎是单线程的，所有节点的tick()方法顺序执行，如果一个tick()阻塞（如长时间sleep），整个树就会被冻结

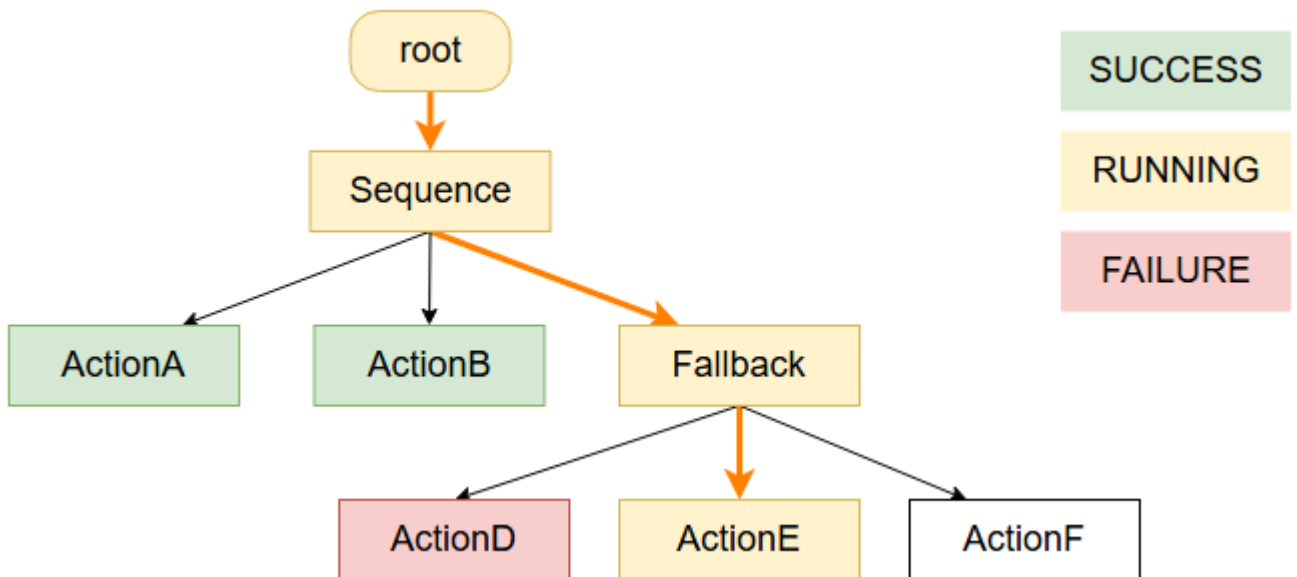
为了实现响应式行为，BT.CPP依赖并发并非并行：即异步动作应尽快返回RUNNING，让树处理其他节点，然后通过后续的tick轮询检查状态

4.异步动作vs同步动作

一个真正的异步动作（Asynchronous Action）应该满足：

- 当被tick时，可能返回RUNNING而不是立即完成
- 必须能够快速中止：当树的逻辑发生变化（如父节点被中断）时，需要能立即停止该动作，并清理资源

同步动作（Synchronous Action）则是一次性执行完毕并返回成功/失败，不会返回RUNNING



5.异步动作的实现：StatefulActionNode

StatefulAction是BT.CPP提供的专门用于实现异步动作的基类。它将动作的生命周期分为三个阶段：

- onStart(): 节点第一次被tick时调用，在这里初始化任务，可以：
 - 立即返回SUCCESS（如果任务无需异步）
 - 返回RUNNING（表示任务已启动，需要后续tick继续）
- onRunning(): 当节点处于RUNNING状态时，每次tick都会调用这个方法。在这里检查任务进度，如果完成则返回SUCCESS/FAILURE，否则继续返回RUNNING
- onHalted(): 当节点被终止（例如父节点不再需要它）时调用，用于清理资源，取消任务

示例：

在下面的例子中，“ActionE”是异步的并处于“运行中”状态；当一个节点处于“运行中”时，通常其父节点也会返回运行中

代码块

```
1  using namespace std::chrono;
2
3  // 使用 StatefulActionNode 作为基类的异步节点示例
4  class SleepNode : public BT::StatefulActionNode
5  {
6  public:
7      SleepNode(const std::string& name, const BT::NodeConfig& config)
8          : BT::StatefulActionNode(name, config)
9      {}
10
11     static BT::PortsList providedPorts()
```

```

12     {
13         // 我们想要休眠的毫秒数
14         return{ BT::InputPort<int>("msec") };
15     }
16
17     NodeStatus onStart() override
18     {
19         int msec = 0;
20         getInput("msec", msec);
21
22         if( msec <= 0 ) {
23             // 无需进入运行中状态
24             return NodeStatus::SUCCESS;
25         }
26         else {
27             // 一旦达到截止时间，我们将返回成功
28             deadline_ = system_clock::now() + milliseconds(msec);
29             return NodeStatus::RUNNING;
30         }
31     }
32
33     /// 在运行中状态下由动作调用的方法
34     NodeStatus onRunning() override
35     {
36         if ( system_clock::now() >= deadline_ ) {
37             return NodeStatus::SUCCESS;
38         }
39         else {
40             return NodeStatus::RUNNING;
41         }
42     }
43
44     void onHalted() override
45     {
46         // 这里无需任何操作...
47         std::cout << "SleepNode interrupted" << std::endl;
48     }
49
50     private:
51         system_clock::time_point deadline_;
52 };

```

在上述代码中：

- 当SleepNode第一次tick时，会执行onStart()方法。如果睡眠时间为0，它会立即返回“成功”；否则返回“运行中”

- 应该在一个循环中持续tick树。浙江调用onRunning()方法，它可能再次返回“运行中”，或者最终返回“成功”
- 另一个节点可能触发halt()信号。在这种情况下，会调用onHalted()方法

6.常见错误：阻塞tick()

实现SleepNode的错误方式如下

代码块

```
1  // 这是该节点的同步版本。可能不是我们想要的。
2  class BadSleepNode : public BT::ActionNodeBase
3  {
4      public:
5          BadSleepNode(const std::string& name, const BT::NodeConfig& config)
6              : BT::ActionNodeBase(name, config)
7          {}
8
9          static BT::PortsList providedPorts()
10         {
11             return{ BT::InputPort<int>("msec") };
12         }
13
14         NodeStatus tick() override
15         {
16             int msec = 0;
17             getInput("msec", msec);
18             // 这个阻塞函数会冻结整个树 :(
19             std::this_thread::sleep_for( milliseconds(msec) );
20             return NodeStatus::SUCCESS;
21         }
22
23         void halt() override
24         {
25             // 因为我冻结了树，没有人能调用这个方法。
26             // 即使这个方法可以执行，我也无法中断 std::this_thread::sleep_for()
27         }
28     };
```

在tick()中直接调用std::this_thread::sleep_for。

这会阻塞整个行为树的执行，导致无法响应其他节点或中止请求

7.多线程的问题

早期版本中，有人尝试为每个异步动作创建新线程。这带来了：

- 线程安全复杂（黑板访问需要同步）
- 很难正确实现halt()来中止线程
- 通常并不需要真正的并行，只需非阻塞的轮询

ThreadedAction基类虽然存在，但推荐避免使用，如果一定要用，需要在线程内部定期检查isHaltRequested()，并短时间休眠，以确保可以快速响应中止。但这样实现起来比StatefulActionNode复杂的多

看看SleepNode：

代码块

```

1  // 这会创建它自己的线程。但在被中止时仍然存在问题
2  class BadSleepNode : public BT::ThreadedAction
3  {
4      public:
5          BadSleepNode(const std::string& name, const BT::NodeConfig& config)
6              : BT::ActionNodeBase(name, config)
7          {}
8
9          static BT::PortsList providedPorts()
10         {
11             return{ BT::InputPort<int>("msec") };
12         }
13
14         NodeStatus tick() override
15         {
16             // 这段代码在其自己的线程中运行，因此树仍在运行。
17             // 这看起来不错，但线程仍然无法被中止
18             int msec = 0;
19             getInput("msec", msec);
20             std::this_thread::sleep_for( std::chrono::milliseconds(msec) );
21             return NodeStatus::SUCCESS;
22         }
23         // halt() 方法无法杀死生成的线程 :(
24     };

```

正确的版本应该是：

代码块

```

1  // 我将在这里创建自己的线程，但并没有充分的理由
2  class ThreadedSleepNode : public BT::ThreadedAction
3  {
4      public:

```

```

5     ThreadedSleepNode(const std::string& name, const BT::NodeConfig&
      config)
6         : BT::ActionNodeBase(name, config)
7     {}
8
9     static BT::PortsList providedPorts()
10    {
11        return{ BT::InputPort<int>("msec") };
12    }
13
14    NodeStatus tick() override
15    {
16        // 这段代码在其自己的线程中运行，因此树仍在运行。
17        int msec = 0;
18        getInput("msec", msec);
19
20        using namespace std::chrono;
21        const auto deadline = system_clock::now() + milliseconds(msec);
22
23        // 定期检查 isHaltRequested()
24        // 并且仅休眠一小段时间 (1 毫秒)
25        while( !isHaltRequested() && system_clock::now() < deadline )
26        {
27            std::this_thread::sleep_for( std::chrono::milliseconds(1) );
28        }
29        return NodeStatus::SUCCESS;
30    }
31
32    // halt() 方法会将 isHaltRequested() 设置为 true
33    // 并停止生成线程中的 while 循环
34 };

```

8.高级示例：客户端/服务器通信

在许多机器人框架中（如ROS ActionLib），动作执行是在另一个进程中完成的，这种通信模式天然契合异步动作：

- 提供要给非阻塞的启动函数（如发送目标）
- 通过轮询或回调检查状态
- 能够预占（preempt）正在执行的动作
- 不需要自己创建线程，只需要在onRunning中检查通信状态

这些操作都不是“阻塞”的，因此不需要创建自己的线程

七、回退节点族

1.回退节点

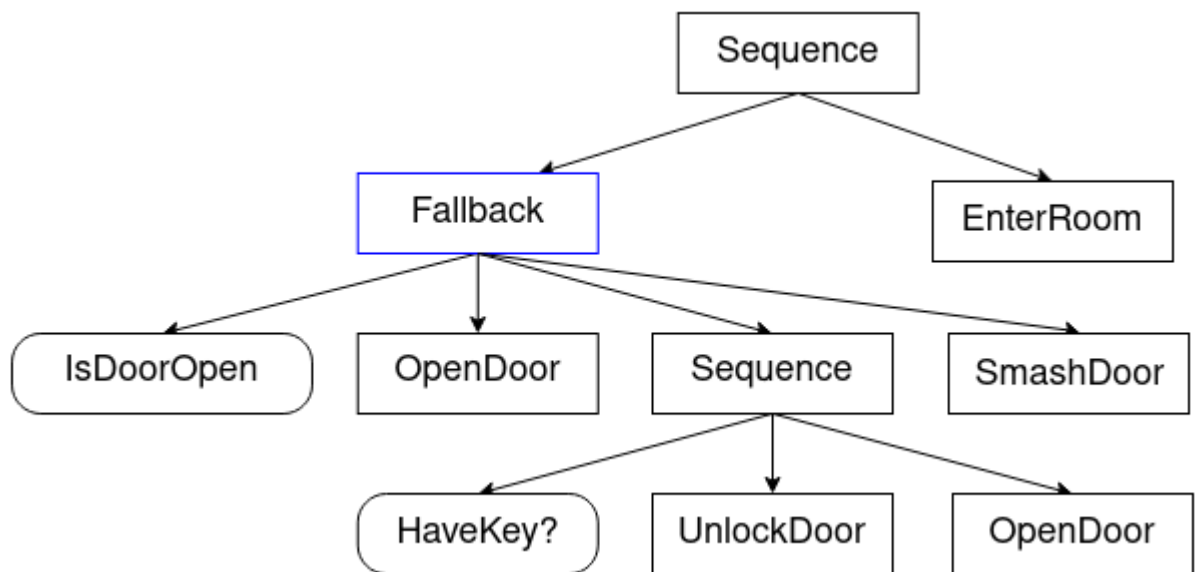
回退节点（也称为“选择节点”）顾名思义，能够表达回退策略，即当一个子节点返回“失败”时下一步该做什么

他会按顺序tick子节点，并且

如果一个子节点返回“失败”，则tick下一个子节点

如果一个子节点返回“成功”，则不再滴答后续子节点，并且该回退节点返回“成功”

如果所有子节点都返回“失败”，那么该回退节点也返回“失败”



这类节点在其他框架中被称为“选择器”或“优先级节点”

其目的是尝试不同的策略，直到找到一种“有效”的策略

目前，该框架提供了三种节点：

- Fallback（回退）
- AsyncFallback（异步回退）
- ReactiveFallback（反应式回退）

它们遵循以下规则：

- 在tick第一个子节点之前，节点状态变为“运行中”
- 如果一个子节点返回“失败”，则回退节点会tick下一个子节点
- 如果最后一个子节点也返回“失败”，则所有子节点都会被中止，并且回退节点返回“失败”
- 如果一个子节点返回“成功”，则回退节点停止并返回“成功”。所有子节点都会被中止

2.同步vs异步

Fallback和AsyncFallback共享相同的逻辑，但在处理子节点之间的转换方式上有所不同：

- Fallback：在树的单次tick中tick所有子节点，当一个子节点返回“失败”时，下一个子节点会在同一次调用中立即被tick
- AsyncFallback：在每个子节点失败后将执行权交换给树，返回“运行中”并发出唤醒信号。这使得回退节点在子节点之间是可中断的，允许树的其他部分（例如，一个ReactiveSequence父节点）在下一个子节点启动之前重新评估条件

对比表：

控制节点类型	子节点返回“运行中”	是否在子节点之间让出执行权
Fallback	再次tick	否
AsyncFallback	再次tick	是
ReactiveFallback	重新开始	否

- “再次tick”意味着下一次回退节点被tick时，同一个子节点会再次被tick。之前已经返回“失败”的兄弟节点不会被再次tick
- “重新开始”意味着整个回退节点从列表第一个子节点重新开始执行

3.Fallback（回退节点）

行为：按顺序尝试子节点，一旦某个子节点返回SUCCESS就停止并返回SUCCESS；如果所有子节点都返回FAILURE，则返回FAILURE，所有子节点在同一个tick内依次执行，不会在子节点之前让出控制权

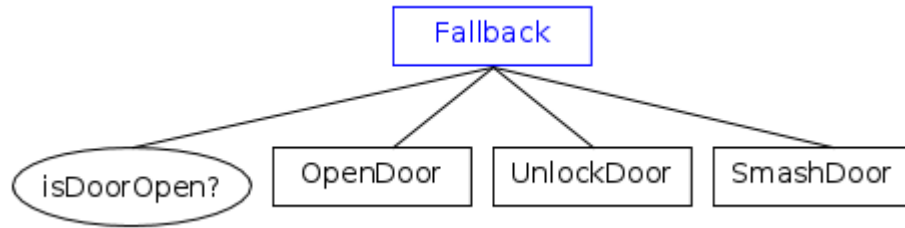
示例：开门策略

代码块

```
1  <Fallback>
2      <IsDoorOpen/>          <!-- 如果门已开，直接成功 -->
3      <OpenDoor/>            <!-- 尝试开门 -->
4      <PickLock/>             <!-- 尝试撬锁 -->
5      <SmashDoor/>            <!-- 最后砸门 -->
6  </Fallback>
```

- 如果门已开，IsDoorOpen返回SUCCESS，Fallback结束，不会尝试后续动作

- 如果门关着，IsDoorOpen返回FAILURE，Fallback一次尝试OpenDoor、PickLock、SmashDoor，直到某个成功或全部失败



4.AsyncFallback

AsyncFallback的行为类似于Fallback，但在每个子节点返回“失败”后将执行权交还给树，它返回“运行中”并发出唤醒信号，允许反应式父节点在下一个子节点被tick之前重新评估条件

代码块

```

1  <ReactiveSequence>
2    <IsRobotHungry/>
3    <AsyncFallback>
4      <FindFoodInBackpack/>
5      <FindNearbyRestaurant/>
6      <OrderFoodDelivery/>
7    </AsyncFallback>
8  </ReactiveSequence>
  
```

在此示例中，IsRobotHungry 会在 AsyncFallback 的每次尝试之间被重新检查。如果在 FindFoodInBackpack 失败后机器人不再饥饿，则在 FindNearbyRestaurant 开始之前，回退节点会被中断。

5.ReactiveFallback（反应式回退）

行为：与Fallback类似，但一旦有某个先前的条件节点从FAILURE变为SUCCESS，它会立即中断当前正在运行的异步子节点，并从第一个子节点重新开始，适用于需要持续监控优先级条件的场景

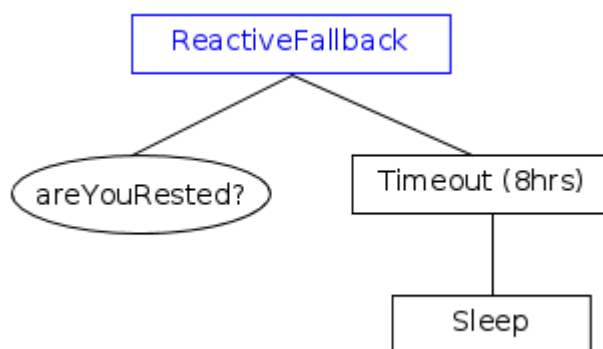
示例：

代码块

```

1  <ReactiveFallback>
2    <AreYouRested/>                                <!-- 条件：是否已休息好 -->
3    <Timeout msec="28800000">                        <!-- 8小时超时 -->
4      <Sleep/>
5    </Timeout>
  
```


- 初始时AreYouRested返回FAILURE（未休息好），Fallback进入Timeout子节点，开始睡眠（异步）
- 在睡眠过程中，每次 tick 都会重新检查 AreYouRested。一旦条件变为 SUCCESS（比如睡眠已足够），ReactiveFallback 会立即中止 Timeout 子节点，并从第一个子节点重新开始。此时 AreYouRested 成功，Fallback 返回 SUCCESS，结束整个睡眠行为。



八、序列节点族

1.基本序列

序列节点是最简单的控制节点，它会一次执行其子节点，并且如果所有子节点都返回成功，那么它本身也会返回成功

序列图

第一次tick会将序列节点设置为“运行中”（橙色状态）

序列节点tick其第一个子节点“开门”，该子节点最终返回“成功”

结果，第二个子节点“行走”以及随后的节点关门会被依次tick

一旦最后一个子节点完成，整个序列节点

序列节点会依次tick其所有节点，只要它们返回“成功”，如果任何子节点返回“失败”，序列将被中止

目前，该框架提供了四种节点：

- Sequence（序列）
- AsyncSequence（异步序列）
- SequenceWithMemory（带记忆的序列）
- ReactiveSequence（反应式序列）

遵循以下规则：

- 在tick第一个子节点之前，节点状态变为“运行中”
- 如果一个子节点返回“成功”，则tick下一个子节点
- 如果最后一个子节点也返回“成功”，则所有子节点都会被中止，并且序列节点返回“成功”

2.同步vs异步

Sequence和AsyncSequence共享相同的逻辑，但在处理子节点之间的转换方式上有所不同：

- Sequence：在树的单词tick中tick所有子节点，当一个子节点返回“成功”时，下一个子节点会在同一次调用中立即被tick
- AsyncSequence：在每个子节点成功后将执行权交还给树，返回“运行中”并发出唤醒信号。这使得序列节点在子节点之间是可中断的，允许树的其他部分（例如，一个ReactiveSequence父节点）在下一个子节点启动之前重新评估条件
- 当序列位于反应式父节点内部，并且需要在每一步之间重新检查条件时，需使用AsyncSequence

对比：

控制节点类型	子节点返回“失败”时	子节点返回“运行中”时	是否在子节点之间让出执行权
Sequence	重启	再次tick	否
AsyncSequence	重启	再次tick	是
ReactiveSequence	重启	重启	否
SequenceWithMemory	再次tick	再次tick	否

- “重启”意味着整个序列从列表的第一个子节点开始执行
- “再次tick”意味着下一次序列被tick时，同一个子节点会被再次tick，之前已经返回“成功”的兄弟节点不会被再次tick

3.Sequence（标准顺序执行）

代码块

```

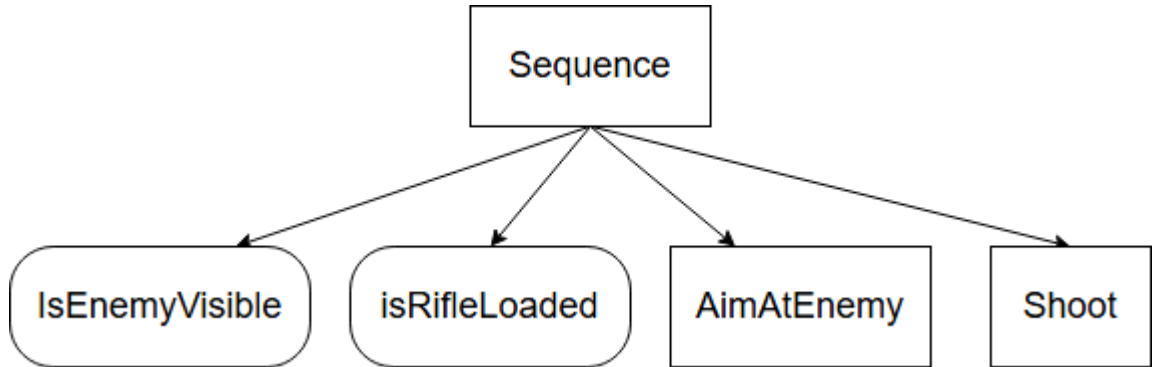
1  <Sequence>
2    <OpenDoor/>
3    <WalkThrough/>
```

```

4     <CloseDoor />
5 </Sequence>

```

- 依次执行：开门->穿过->关门
- 如果某一步失败，整个序列立即失效，不再执行后续步骤，并且下次tick会从头开始



4.Sequence与ReactiveSequence的区别

普通Sequence

- 行为：按顺序执行子节点，只执行一次，如果一个子节点返回RUNNING，下次tick会继续执行该子节点（不重新执行前面的子节点）
- 特点：一旦开始执行要给异步动作，他就不会重新检查前置条件（如电池状态），知道该动作完成或失败

示例树：

代码块

```

1  <root BTCPP_format="4">
2      <BehaviorTree>
3          <Sequence>
4              <BatteryOK />
5              <SaySomething    message="mission started..." />
6              <MoveBase        goal="1;2;3" />
7              <SaySomething    message="mission completed!" />
8          </Sequence>
9      </BehaviorTree>
10 </root>

```

主程序：

代码块

```

1  int main()
2  {

```

```

3     BT::BehaviorTreeFactory factory;
4     factory.registerSimpleCondition("BatteryOK", std::bind(CheckBattery));
5     factory.registerNodeType<MoveBaseAction>("MoveBase");
6     factory.registerNodeType<SaySomething>("SaySomething");
7
8     auto tree = factory.createTreeFromText(xml_text);
9
10    // 此处, 我们不使用 tree.tickWhileRunning(),
11    // 而是采用我们自己的循环
12    std::cout << "--- ticking\n";
13    auto status = tree.tickOnce();
14    std::cout << "--- status: " << toString(status) << "\n\n";
15
16    while(status == NodeStatus::RUNNING)
17    {
18        // 休眠以避免忙循环
19        // 不要使用其他休眠函数!
20        // 较小的休眠时间是可以的, 这里我们使用较大的值只是为了在控制台上少输出一些消息
21        tree.sleep(std::chrono::milliseconds(100));
22
23        std::cout << "--- ticking\n";
24        status = tree.tickOnce();
25        std::cout << "--- status: " << toString(status) << "\n\n";
26    }
27
28    return 0;
29 }

```

执行流程（输出）：

代码块

```

1    --- ticking
2    [ Battery: OK ]
3    Robot says: mission started...
4    [ MoveBase: 发送请求 ]. 目标: x=1.0 y=2.0 theta=3.0
5    --- status: RUNNING
6
7    --- ticking
8    --- status: RUNNING
9
10   --- ticking
11   [ MoveBase: 完成 ]
12   Robot says: mission completed!

```

```
13    --- status: SUCCESS
```

当调用executeTick()时，MoveBase第一次和第二次返回RUNNING，最终第卅那次返回SUCCESS，BatteryOK只执行了一次

ReactiveSequence（反应式序列）

- 行为：和Sequence类似，但每当一个子节点返回RUNNING时，下一次tick会从序列的第一个子节点重新开始执行
- 特点：它会在每次tick时重新评估前置条件，如果条件不再满足（例如电池变为FAILURE），则可以立即中止正在运行的异步动作并退出

示例树：

代码块

```
1    <root>
2        <BehaviorTree>
3            <ReactiveSequence>
4                <BatteryOK/>
5                <Sequence>
6                    <SaySomething    message="mission started..." />
7                    <MoveBase        goal="1;2;3"/>
8                    <SaySomething    message="mission completed!" />
9                </Sequence>
10           </ReactiveSequence>
11       </BehaviorTree>
12   </root>
```

执行流程：

代码块

```
1    --- ticking
2    [ Battery: OK ]
3    Robot says: mission started...
4    [ MoveBase: 发送请求 ]. 目标: x=1.0 y=2.0 theta=3.0
5    --- status: RUNNING
6
7    --- ticking
8    [ Battery: OK ]
9    --- status: RUNNING
10
11   --- ticking
12   [ Battery: OK ]
```

```
13 [ MoveBase: 完成 ]
14 Robot says: mission completed!
15 --- status: SUCCESS
```

区别

在ReactiveSequence中，BatteryOK每次tick都会被重新执行，如果策略BatteryOK返回FAILURE（比如电量突然低了），ReactiveSequence会立即停止内部序列（并调用内部的动作onHalted()），然后整体返回失败。这样就能实现对外部条件变化的快速响应

5.AsyncSequence（让出控制权）

代码块

```
1 <ReactiveSequence>
2   <IsEnemyVisible/>
3   <AsyncSequence>
4     <AimWeapon/>
5     <FireWeapon/>
6     <ReloadWeapon/>
7   </AsyncSequence>
8 </ReactiveSequence>
```

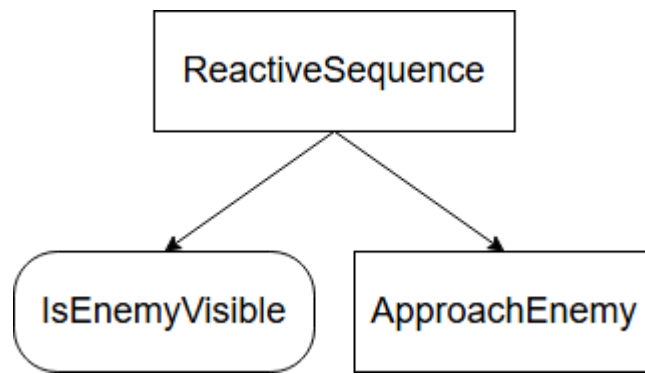
- 每次成功执行一个子节点后，AsyncSequence会返回RUNNING，让树有机会重新检查IsEnemyVisible
- 如果敌人消失，这个序列可以被中断，避免继续执行后续动作

6.ReactiveSequence（持续监控）

代码块

```
1 <ReactiveSequence>
2   <IsBatteryOK/>           <!-- 条件节点 -->
3   <MoveToGoal/>            <!-- 异步动作 -->
4   <UnloadCargo/>
5 </ReactiveSequence>
```

- 每次tick都会从第一个子节点开始执行
- 如果电池不足（IsBatteryOK返回FAILURE），则整个序列失效，MoveToGoal会被中止
- 适合需要持续监控前置条件的场景



7.SequenceWithMemory（带记忆的顺序执行）

特点：一旦某个子节点成功，它就不会被再次执行；即使后续子节点失效，也不会重置已成功
的子节点。适用于需要“一次性任务”且不重复的场景

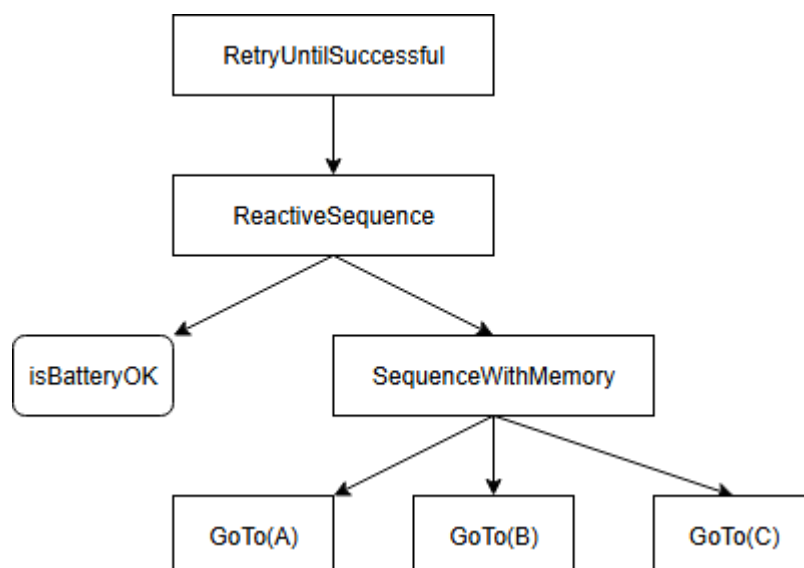
示例：巡逻机器人依次访问3个点

代码块

```

1  <SequenceWithMemory>
2    <GoToLocation A="10,20"/>
3    <GoToLocation B="30,40"/>
4    <GoToLocation C="50,60"/>
5  </SequenceWithMemory>
  
```

- 假设GoToLocationA成功，机器人记住了A点已完成
- 接下来尝试B点，如果B点失败（例如路径阻塞），SequenceWithMemory不会重新执行A点，而是继续尝试B点（下次tick仍然在B点重试），直到B点成功
- B点成功后，再执行C点，如果C点失败，同样会持续重试C点，不会回退到A或B
- 当所有子节点都成功时，整个序列返回SUCCESS



九、并行节点

并行节点会并发执行其所有子节点，但并非在单独的线程中执行，在树的首次tick中，所有子节点都会按顺序被tick，但多个子节点可以同时处于“运行中”状态

注意：

- “并行”值得是多个子节点可以同时处于“运行中”状态这一事实。子节点仍然在同一线程内按顺序被tick。要实现真正的多线程执行，子节点内部必须是异步节点
- 并行节点是唯一允许同时有多个子节点处于“运行中”状态的节点

该框架提供了两种节点：

- Parallel（并行）
- ParallelAll（全并行）

1.Parallel

当达到成功或失败的阈值时，该节点完成。任何剩余处于“运行中”的子节点将被中止

端口	类型	默认值	描述
success_count	输入端口<int>	-1	需要成功才能返回“成功”的子节点数量
failure_count	输入端口<int>	1	需要失败才能返回“失败”的子节点数量

阈值支持Python风格的副索引：-1等同于子节点的总数。例如：对于4个子节点，success_count="-1"意味着所有4个子节点都必须成功。

默认行为（success_count=-1，failure_count=1）：所有子节点都必须成功；如果任何一个子节点失败，该节点返回“失败”，和Sequence类似，但允许孩子同时运行

代码块

```
1 <Parallel success_count="2" failure_count="2">
2   <ActionA/>
3   <ActionB/>
4   <ActionC/>
5 </Parallel>
```

在这个包含3个子节点的示例中：

- 如果有2个子节点返回“成功”（在2个子节点失败之前），该节点返回“成功”

- 如果有2个子节点返回“失败”（在2个子节点成功之前），该节点返回“失败”
- 当达到任一阈值时，剩余处于“运行中”的子节点将被终止

2.ParallelAll

与Parallel不同，ParallelAll节点总是执行所有子节点直至完成。它永远不会提前中止子节点。当需要所有副作用都完成时，非常有用

端口	类型	默认值	描述
max_failure	输入端口<int>	1	在返回“失败”之前允许的子节点最大失败次数

- 如果失败次数小于max_failures，则返回“成功”
- 如果失败次数等于或超过max_failures，则返回“失败”
- 使用max_failures="-1"（子节点数量）将忽略子节点结果，始终返回“成功”

代码块

```

1  <ParallelAll max_failures="2">
2    <ActionA/>
3    <ActionB/>
4    <ActionC/>
5  </ParallelAll>

```

Parallel与ParallelAll对比

特性	Parallel	ParallelAll
提前终止	是（达到阈值时中止子节点	否（总是运行所有子节点）
成功阈值	可配置(succeed_count)	所有未失败子节点
失败阈值	可配置(failure_count)	可配置（max_failures）
适用场景	竞争条件，N选M成功	所有任务都必须尝试完成

Parallel示例：

机器人同时尝试用视觉和红外定位，只要有一种定位成功就继续，或者多个备用方案，只要其中一个完成即成功

ParallelAll示例：

机器人在关闭前需要同时执行保存日志、关闭点击、释放资源，这些动作无论成败都需尝试执行，且不允许因某个失败而跳过其他任务

十、条件控制与Switch节点

1.条件控制节点

这些控制节点根据条件（第一个子节点）的结果来选择执行哪个子节点

目前，该框架提供了两种节点：

IfThenElse

WhileDoElse

这两种节点都必须恰好有2个或3个子节点

1.1IfThenElsea：一次性条件判断

IfThenElse是一个非反应式条件节点

第一个子节点条件是（"if"）

如果条件返回“成功”，则执行第二个子节点（“then”）

如果条件返回“失败”，则返回第三个子节点（可选，“else”）

如果只提供了2个子节点且条件返回“失败”，则该节点返回“失败”（相当于AlwaysFailure）作为第三个子节点

条件仅评估一次，如果第二个或第三个子节点返回“运行中”，则在后续tick中不会重新评估条件

```
<IfThenElse>
  <IsDoorOpen/>    <!-- 条件 -->
  <WalkThrough/>    <!-- 则分支 -->
  <TryAnotherPath/> <!-- 否则分支（可选） -->
</IfThenElse>
```

1.2.WhileDoElse：反应式循环判断

WhileDoElse是IfThenElse的反应式变体。它会在每次tick时重新评估条件

第一个子节点是条件，在每次tick时评估（“while”）

当条件返回“成功”时，执行第二个子节点（“do”）

当条件返回“失败”时，执行第三个子节点（可选，“esle”）

如果第二个或第三个子节点处于“运行中”状态，并且条件结果发生变化，则在切换到另一个分支之前，正在运行的子节点会被中止

1.3.两者之间的区别

特性	IfThenElse	WhileDoElse
条件的评估时机	仅一次（进入时）	每次tick都重新评估
反应式切换	不支持	支持（条件变化时自动中止并切换）
适用场景	一次性决策	持续监控、反应式行为
分支执行期间	不重新评估条件	持续评估，条件变化时切换

2.Switch节点

Switch节点等同于switch语句：它根据黑板变量的值选择执行哪个子节点

可用的变体有：Switch2、Switch3、Switch4、Switch5、Switch6，其中数字表示支持的case分支数量

一个SwitchN节点必须恰好有N+1个子节点：N个case分支加上一个默认分支（始终是最后一个子节点）

端口	类型	描述
variable	输入端口<string>	用于与case值进行比较的黑板变量
case_1	输入端口<string>	与第一个子节点匹配的值
case_2	输入端口<string>	与第二个子节点匹配的值
...	...	最多N个附加case
case_N	输入端口<string>	与第N个子节点匹配的值

变量值按顺序与每个case_N字符串进行比较。执行第一个匹配项对应的子节点。如果没有case匹配，则执行最后一个子节点（默认分支）

比较支持字符串、整数和双精度浮点数。通过ScriptingEnumsRegistry注册的枚举值也受支持

```

代码块
1 <Switch3 variable="{robot_state}" case_1="IDLE" case_2="WORKING"
  case_3="ERROR">
2   <HandleIdle/>           <!-- 当 robot_state == "IDLE" 时执行 -->
3   <HandleWorking/>        <!-- 当 robot_state == "WORKING" 时执行 -->
4   <HandleError/>          <!-- 当 robot_state == "ERROR" 时执行 -->
5   <HandleUnknownState/>   <!-- 默认: 当没有 case 匹配时执行 -->
6 </Switch3>

```

每次tick都会重新检查变量。如果当前正在运行的子节点所属的case不再匹配（因为变量值改变了），节点会先中止（halt）当前运行的子节点，然后立即切换到新匹配的case（或default）分支

这让Switch能够对变量的变化做出即时响应，而不需要等待当前动作完成

同样的行为可以通过使用多个序列、回退和条件来实现，但Switch节点更加简洁和易读

十一、装饰节点大全

装饰器是一种必须只有一个子节点的节点

由装饰器决定是否、何时以及多少次tick其子节点

1.Inverter（反转节点）

tick子节点一次，如果子节点失败则返回成功，如果子节点成功则返回失败

如果子节点返回“运行中”，则该节点也返回“运行中”

代码块

```

1 <Inverter>
2   <IsDoorClosed/>    <!-- 假设门关着时返回 SUCCESS，取反后变成 FAILURE -->
3 </Inverter>

```

2.ForceSuccess（强制成功节点）

如果子节点返回“运行中”，则该节点也返回“运行中”

否则，它总是返回成功

代码块

```

1 <ForceSuccess>
2   <OpenDoor/>         <!-- 无论开门成功还是失败，最终都返回 SUCCESS -->
3 </ForceSuccess>

```

3.ForceFailure（强制失败节点）

如果子节点返回“运行中”，则该节点也返回“运行中”

否则，它总是返回失败

代码块

```
1 <ForceFailure>
2   <OpenDoor/>      <!-- 无论开门成功还是失败，最终都返回 FAILURE -->
3 </ForceFailure>
```

4.Repeat（重复节点）

只要子节点返回成功，就tick子节点，最多N次

端口	类型	默认值	描述
num_cycles	输入端口<int>	（必需）	重复次数，使用-1表示无限循环

- 如果所有N次重复都成功完成，则返回成功
- 如果子节点返回失败，则立即返回失败（循环终端）
- 如果子节点返回“运行中”，则返回“运行中”；计数器不会递增，并且在下一次tick时继续执行同一次迭代
- 如果子节点返回“已跳过”，子节点会被重置，计数器不会递增

代码块

```
1 <Repeat num_cycles="3">
2   <ClapYourHandsOnce/>
3 </Repeat>
```

5.TetryUntilSuccessful（重试至成功节点）

只要子节点返回失败，就tick子节点，最多N次

端口	类型	默认值	描述
num_attempts	输入端口<int>	（必需）	尝试次数。使用-1表示无限重试

- 如果子节点返回成功，则立即返回成功（循环中断）
- 如果所有N次禅师都耗尽，则返回失败
- 如果子节点返回“运行中”，则返回“运行中”；尝试计数器不会递增，并且在下一次tick时继续执行同一迭代
- 如果子节点返回“已跳过”，子节点会被重置，并返回“已跳过”

代码块

```
1 <RetryUntilSuccessful num_attempts="3">
2   <OpenDoor/>
3 </RetryUntilSuccessful>
```

6.KeepRunningUntilFailure（保持运行直至失败节点）

KeepRunningUntilFailure节点总是返回失败（子节点失败）或运行中（子节点成功或运行中）

代码块

```
1 <KeepRunningUntilFailure>
2   <MonitorBattery/> <!-- 假设 MonitorBattery 一直返回 SUCCESS，则
   KeepRunningUntilFailure 永远返回 RUNNING，直到电池耗尽返回 FAILURE -->
3 </KeepRunningUntilFailure>
```

7.Delay（延迟节点）

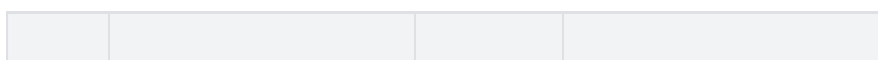
在指定的时间过去后tick子节点，延迟时间通过输入端口delay_msec指定。如果子节点返回“运行中”，则该节点也返回“运行中”，并将在Delay节点的下一次tick时tick子节点。否则，返回子节点的状态

代码块

```
1 <Delay delay_msec="3000">
2   <SaySomething message="Delay finished!"/>
3 </Delay>
```

8.Timeout（超时节点）

如果运行中的子节点运行时间超过给定持续时间，则中止它。这与Delay相反：Delay在tick子节点之前等待，而Timeout则会中断耗时过长的子节点



端口	类型	默认值	描述
msec	输入端口<unsigned>	(必需)	超时持续时间（毫秒）

- 如果子节点在超市之前完成（成功或失败），则返回其状态
- 如果超时时间到期时子节点仍处于“运行中”状态，则它会被中止，并返回失败

代码块

```
1 <Timeout msec="5000">
2     <KeepYourBreath/>
3 </Timeout>
```

提示：

将Timeout与RetryUntilSuccessful结合使用，可以形成一个强大的带超时的重试模式

代码块

```
1 <!-- 重试超时模式：最多尝试 3 次，每次超时 5 秒 -->
2 <RetryUntilSuccessful num_attempts="3">
3     <Timeout msec="5000">
4         <LongRunningAction/>
5     </Timeout>
6 </RetryUntilSuccessful>
7
8 <!-- 仅在电池电量高于 20 时执行，否则返回 FAILURE -->
9 <Precondition if="battery_level > 20" else="FAILURE">
10     <MoveToGoal/>
11 </Precondition>
```

9.RunOnce（单次执行节点）

当你想只执行子节点一次时，使用RunOnce节点，如果子节点是异步的，它会持续tick直到返回成功或失败

在首次执行后，可以将输入端口then_skip的值设置为：

- TRUE（默认），该节点将来会被跳过
- FALSE，同步返回子节点返回的状态，永久如此

代码块

```
1 <RunOnce then_skip="true">
2     <InitializeRobot/>
```

10.Precondition（前置条件节点）

Precondition装饰器在tick其子节点之前评估一个脚本条件

端口	类型	默认值	描述
if	输入端口<std::string>	(必需)	要评估的脚本条件
else	输入端口<NodeStatus>	FAILURE	如果条件为假，则返回状态

- 如果条件为真，则tick子节点
- 如果条件为假，则节点返回else中指定的状态
- 一旦子节点开始（返回“运行中”），直到子节点完成，条件才会被重新评估

代码块

```
1 <Precondition if="battery_level > 20" else="FAILURE">
2   <MoveToGoal/>
3 </Precondition>
```

如果需要在子节点运行时，在每次tick时都检查条件（例如，在条件发生拜年话时中断正在运行的动作），可以使用else="RUNNING"：

代码块

```
1 <Precondition if="battery_ok" else="RUNNING">
2   <LongRunningAction/>
3 </Precondition>
```

使用else="RUNNING"，如果条件变为假，装饰器将返回“运行中”而不是“失败”，从而允许树继续tick并重新检查条件

11.SubTree（子树节点）

（参阅“使用子树组合行为”）

需要在C++中注册其他的装饰器

空队列将返回成功

例如，使用 `factory.registerNodeType<ConsumeQueue<Pose2D>>("ConsumeQueue");` 进行注册。可参考 `t18_waypoints.cpp`。

12.SimpleDecoratorNode（简单装饰器节点）

使用 `void BehaviorTreeFactory::registerSimpleDecorator("MyDecorator", tick_function, ports)` 注册一个简单的装饰器节点，该函数内部使用 `SimpleDecoratorNode`，其中 `tick_function` 是一个签名为 `std::function<NodeStatus(NodeStatus, TreeNode&)>` 的函数，而 `ports` 是一个 `PortsList` 类型的变量。

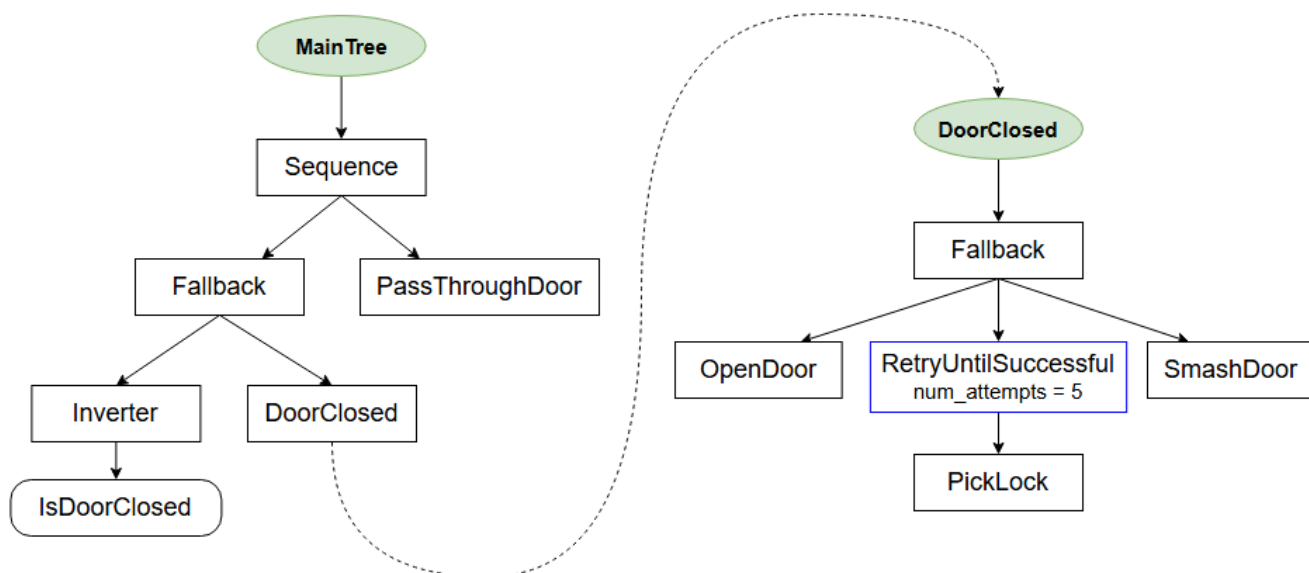
十二、子树与模块化开发

1.使用子树的原因

我们可以通过将较小且可重用的行为插入到较大的行为中来构建大规模行为，换句话说，我们希望创建层次化的行为树，并使我们的树具有可组合性，可以通过在XML定义多个树，并使用 `SubTree` 节点将一个树包含到另一个树中来实现

2.组合子树

2.1.CrossDoor行为



这是一个同时使用Decorators（装饰节点）和Fallback（回退节点）的实际示例
定义了两棵行为树：

代码块

```
1 <root BTCPP_format="4">
```

```

2
3     <BehaviorTree ID="MainTree">
4         <Sequence>
5             <Fallback>
6                 <Inverter>
7                     <IsDoorClosed/>
8                 </Inverter>
9                 <SubTree ID="DoorClosed"/>
10            </Fallback>
11            <PassThroughDoor/>
12        </Sequence>
13    </BehaviorTree>
14
15    <BehaviorTree ID="DoorClosed">
16        <Fallback>
17            <OpenDoor/>
18            <RetryUntilSuccessful num_attempts="5">
19                <PickLock/>
20            </RetryUntilSuccessful>
21            <SmashDoor/>
22        </Fallback>
23    </BehaviorTree>
24
25 </root>

```

主树逻辑：

- 首先是一个Fallback（回退节点），它有两个子节点
 - 第一个子节点是Inverter（取反器），里面是条件节点IsDoorClosed（询问“门是否关着”）

所以这个组合的意思是：如果门不是关着的（即门是开着的），则取反器返回SUCCESS，Fallback就会成功，跳过第二个子节点
 - 第二个子节点是SubTree，引用了ID="DoorClosed"的子树，只有当门是关着的（第一个子节点失败）时，才会执行这个子树

子树的逻辑（门关着时执行）：

这是一个Fallback，依次尝试三种方法开门

- 先尝试OpenDoor（正常开门）。如果成功，Fallback结束
- 如果开门失败，就执行RetryUntilSuccessful装饰节点，它会重复执行子节点PickLock最多五次，直到成功
- 如果撬锁也失败，最后尝试SmashDoor（砸门），一定会成功

只要其中任何一种方法成功，子树就返回SUCCESS，否则返回FAILURE（但砸门总是成功，所以实际上子树总会成功）

2.2C++代码，动作的实现和注册

代码块

```
1  class CrossDoor
2  {
3  public:
4      void registerNodes(BT::BehaviorTreeFactory& factory);
5
6      // 如果 _door_open != true 则返回 SUCCESS
7      BT::NodeStatus isDoorClosed();
8
9      // 如果 _door_open == true 则返回 SUCCESS
10     BT::NodeStatus passThroughDoor();
11
12     // 尝试 5 次后，将打开一扇锁着的门
13     BT::NodeStatus pickLock();
14
15     // 如果门锁着则返回 FAILURE
16     BT::NodeStatus openDoor();
17
18     // 总是能打开门
19     BT::NodeStatus smashDoor();
20
21 private:
22     bool _door_open    = false;
23     bool _door_locked  = true;
24     int _pick_attempts = 0;
25 };
26
27 // 辅助方法，使注册对用户更简单
28 void CrossDoor::registerNodes(BT::BehaviorTreeFactory &factory)
29 {
30     factory.registerSimpleCondition(
31         "IsDoorClosed", std::bind(&CrossDoor::isDoorClosed, this));
32
33     factory.registerSimpleAction(
34         "PassThroughDoor", std::bind(&CrossDoor::passThroughDoor,
35                                     this));
36
37     factory.registerSimpleAction(
38         "OpenDoor", std::bind(&CrossDoor::openDoor, this));
39
40     factory.registerSimpleAction(
41         "SmashDoor", std::bind(&CrossDoor::smashDoor, this));
42 }
```

```

40         "PickLock", std::bind(&CrossDoor::pickLock, this));
41
42     factory.registerSimpleCondition(
43         "SmashDoor", std::bind(&CrossDoor::smashDoor, this));
44 }
45
46 int main()
47 {
48     BehaviorTreeFactory factory;
49
50     CrossDoor cross_door;
51     cross_door.registerNodes(factory);
52
53     // 在这个例子中，单个 XML 包含多个 <BehaviorTree>
54     // 要确定哪一个是“主树”，我们应该先注册 XML，
55     // 然后使用其 ID 分配特定的树
56     factory.registerBehaviorTreeFromText(xml_text);
57     auto tree = factory.createTree("MainTree");
58
59     // 辅助函数，用于打印树
60     printTreeRecursively(tree.rootNode());
61
62     tree.tickWhileRunning();
63
64     return 0;
65 }

```

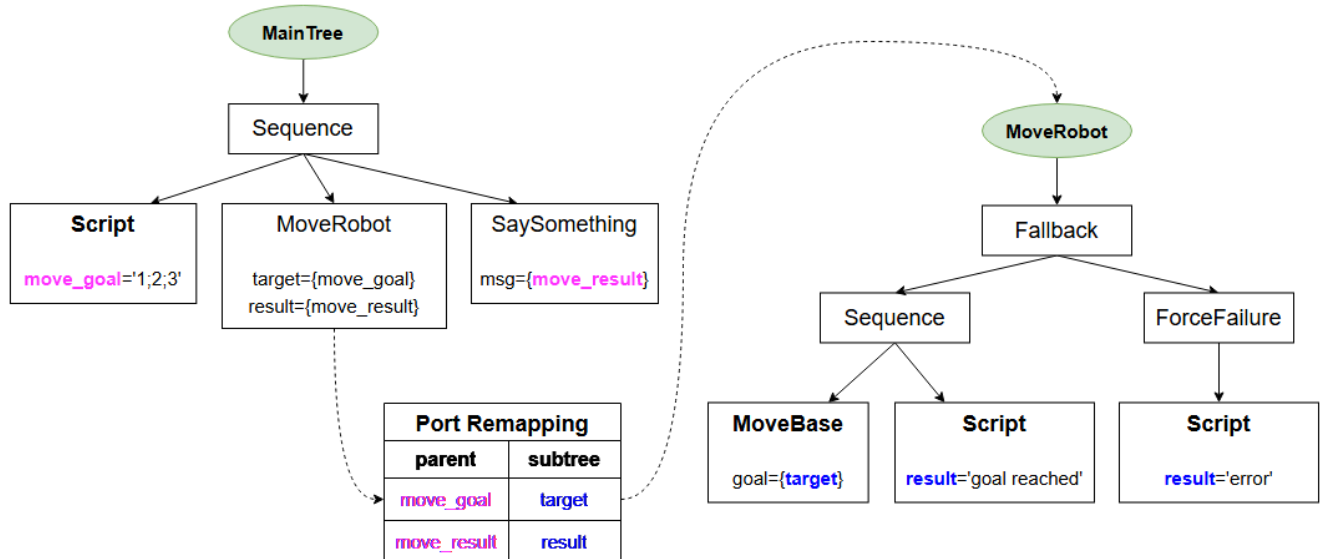
整体行为：

- 初始状态：门是关着的（_door_open=false），并且锁着（_door_locked=true），撬锁次数为0
- 第一次tick：
 - 主树的Fallback先执行Inverter(IsDoorClosed)：因为门关着，IsDoorClosed返回SUCCESS，取反后变成FAILURE，所以Fallback执行第二个子节点SubTree
 - 进入子树DoorClosed
 - 先尝试OpenDoor：门锁着，返回FAILURE
 - 然后执行TetryUntilSuccessful(PickLock)：PickLock第一次尝试失败（因为还没到3次），返回FAILURE，重试节点会再次tick它，这样经过三次tick，PickLock成功（假设第三次成功），子树返回SUCCESS
 - 子树成功后，主树的Fallback返回SUCCESS，接着执行PassThroughDoor，成功穿过，主树返回SUCCESS
- 如果在撬锁过程中门的状态被外部改变（比如锁被解开），PickLock可能更早成功

- 如果OpenDoor成功（例如门没锁），子树会立即返回成功，不会尝试撬锁或砸门

3.子树端口重映射

当一棵树包含子树时，每个子树拥有自己独立的的黑板，为了避免不同树的黑板键名冲突，需要通过重映射将主树的端口与子树的端口显式连接起来。



3.1.端口重映射的语法

在<SubTree>标签中，通过属性将主树的端口（或黑板条目）传递给子树的端口

代码块

```
1 <SubTree ID="MoveRobot" target="{move_goal}" result="{move_result}" />
```

- target="{move_goal}" 表示：子树的 target 输入端口将读取主树黑板中 move_goal 键的值。
- result="{move_result}" 表示：子树的 result 输出端口将写入主树黑板中 move_result 键的值（子树内部通过 setOutput 写入自己的 result 端口，但实际数据会被映射到主树的 move_result）。

这里的 target 和 result 是子树内部定义的端口名，move_goal 和 move_result 是主树黑板上的键名。

3.2.示例

代码块

```
1 <root BTCPP_format="4">
2   <BehaviorTree ID="MainTree">
3     <Sequence>
4       <!-- 在主树黑板上写入一个键 move_goal, 值为字符串 "1;2;3" -->
```

```

5         <Script code=" move_goal='1;2;3' " />
6
7         <!-- 调用子树 MoveRobot, 并将主树的 move_goal 映射到子树的
target,
8             将子树的 result 映射到主树的 move_result -->
9         <SubTree ID="MoveRobot" target="{move_goal}" result="
{move_result}" />
10
11         <!-- 读取主树的 move_result 并打印 -->
12         <SaySomething message="{move_result}"/>
13     </Sequence>
14 </BehaviorTree>
15
16 <BehaviorTree ID="MoveRobot">
17     <Fallback>
18         <Sequence>
19             <!-- 子树的 target 输入端口接收来自主树的值 (1;2;3) -->
20             <MoveBase goal="{target}"/>
21             <!-- 向子树的 result 输出端口写入字符串 "goal reached",
22                 该值会被映射到主树的 move_result -->
23             <Script code=" result:='goal reached' " />
24         </Sequence>
25         <ForceFailure>
26             <Script code=" result:='error' " />
27         </ForceFailure>
28     </Fallback>
29 </BehaviorTree>
30 </root>

```

执行流程：

- 主树执行 <Script>, 将字符串 "1;2;3" 写入主树黑板的 move_goal 键。
- 进入子树 MoveRobot:
 - 子树的 target 端口被重映射到主树的 move_goal, 因此子树内部读取 target 时, 实际上得到的是 "1;2;3"。
 - 子树执行 <MoveBase goal="{target}"/>, 用该值作为导航目标。
 - 如果导航成功, 接着执行 <Script code=" result:='goal reached' " />, 将字符串 "goal reached" 写入子树的 result 端口。由于该端口被重映射到主树的 move_result, 实际上数据被写入了主树黑板的 move_result 键。
 - 如果导航失败, Fallback 的第二个分支会执行 <Script code=" result:='error' " />, 将 "error" 写入主树的 move_result。

- 子树返回后，主树执行 `<SaySomething message="{move_result}"/>`，从主树黑板读取 `move_result` 并打印。

3.3通过C++代码观察黑板内容

代码块

```
1  int main()
2  {
3      BT::BehaviorTreeFactory factory;
4
5      factory.registerNodeType<SaySomething>("SaySomething");
6      factory.registerNodeType<MoveBaseAction>("MoveBase");
7
8      factory.registerBehaviorTreeFromText(xml_text);
9      auto tree = factory.createTree("MainTree");
10
11     // 持续滴答直到结束
12     tree.tickWhileRunning();
13
14     // 让我们查看一些关于黑板当前状态的信息
15     std::cout << "\n----- 第一个黑板 -----" << std::endl;
16     tree.subtrees[0]->blackboard->debugMessage();
17     std::cout << "\n----- 第二个黑板 -----" << std::endl;
18     tree.subtrees[1]->blackboard->debugMessage();
19
20     return 0;
21 }
22
23 /* 预期输出:
24
25 ----- 第一个黑板 -----
26 move_result (std::string)
27 move_goal (Pose2D)
28
29 ----- 第二个黑板 -----
30 [result] 重映射到父树的端口 [move_result]
31 [target] 重映射到父树的端口 [move_goal]
32
33 */
```

- 主树黑板（First BB）中有两个键：`move_goal` 和 `move_result`。
- 子树黑板（Second BB）显示，它的端口 `result` 和 `target` 被重映射到了父树的对应端口，因此子树黑板本身不存储这些键的值，而是通过重映射访问父黑板。

3.4子树模型与自动重映射

当在多个地方使用同一个子树时，需要复制粘贴同样冗长的XML标签，例如以下情况：

代码块

```
1 <SubTree ID="MoveRobot" target="{move_goal}" frame="world" result="{error_code}" />
```

不希望每次复制粘贴target、frame和result这三个XML属性，除非值确实不同

3.4.1为子树定义默认端口映射

为了避免这种情况，可以在<TreeNodesModel>中定义他们的默认值

代码块

```
1 <TreeNodesModel>
2   <SubTree ID="MoveRobot">
3     <input_port name="target" default="{move_goal}" />
4     <input_port name="frame" default="world" />
5     <output_port name="result" default="{error_code}" />
6   </SubTree>
7 </TreeNodesModel>
```

可以在XML中的<TreeNodeModel>中为这个子树声明端口的默认重映射值，就像给普通节点端口设置默认值一样

- 这里声明了MoveRobot子树的三个端口，并指定了默认应该重映射到哪个黑板键（或默认字符串）
- 之后调用该子树时，如果不显式指定这些属性，就会自动使用默认映射

调用时覆盖默认：

代码块

```
1 <SubTree ID="MoveRobot" frame="map" />
```

- 只覆盖了frame，将其映射到黑板键map（注意这里frame是子树的端口名，"map"是黑板键还是字符串取决于端口类型）
- 其他端口target和result仍然使用默认映射{move_goal}和{error_code}

3.4.2.自动重映射

当子树中的端口名与主树中想要使用的黑板键名完全相同时，手动写port="{port}"会显得很啰嗦，_autoremap="true"就是用来解决这个问题的

原始写法：

代码块

```
1 <SubTree ID="MoveRobot" target="{target}" frame="{frame}" result="{result}" />
```

每个端口都映射到同名的黑板键，啰马重复

使用_autoremap：

代码块

```
1 <SubTree ID="MoveRobot" _autoremap="true" />
```

- 这告诉子树：对于它内部使用的所有端口，默认都去主树黑板上找同名的键进行映射
- 即子树内部的target端口自动映射到主树的{target}，frame映射到{frame}，result映射到{result}

部分覆盖+autoremap：

代码块

```
1 <SubTree ID="MoveRobot" _autoremap="true" frame="world" />
```

- 先应用自动重映射（target->{target}，result->{result}），但frame被显式覆盖为固定字符串"world"（而不是黑板键{frame}）
- 这提供了最大灵活性，大部分端口自动匹配，个别端口特殊处理

3.4.3.关于私有条目的约定

如果一个子树内部有某些黑板条目不像被自动重映射影响（即希望它们是子树私有的，不与父树共享），可以在这些条目的名字前加下划线_

代码块

```
1 <SubTree ID="MoveRobot" _autoremap="true" frame="world" />
```

- 当_autoremap="true"时，所有以下划线开头的条目不会被自动重映射，它们依然保留在子树自己的黑板中，不会去父树查找同名键

- 这是一种简单的“私有变量”约定

4.管理多个XML文件

- 随着项目变大，将所有行为树定义放在一个XML文件中会变得臃肿，难以维护
- 将每个子树独立成文件，可以实现模块化和复用

方法一：手动加载所有XML文件

子树1: subtree_A.xml:

代码块

```
1  <root>
2      <BehaviorTree ID="SubTreeA">
3          <SaySomething message="Executing Sub_A" />
4      </BehaviorTree>
5  </root>
```

子树2: subtree_B.xml

代码块

```
1  <root>
2      <BehaviorTree ID="SubTreeB">
3          <SaySomething message="Executing Sub_B" />
4      </BehaviorTree>
5  </root>
```

主树main_tree.xml，包含其他两个文件

代码块

```
1  <root>
2      <BehaviorTree ID="MainTree">
3          <Sequence>
4              <SaySomething message="starting MainTree" />
5              <SubTree ID="SubTreeA" />
6              <SubTree ID="SubTreeB" />
7          </Sequence>
8      </BehaviorTree>
9  </root>
```

通过C++代码进行加载：

```

代码块
1 int main()
2 {
3     BT::BehaviorTreeFactory factory;
4     factory.registerNodeType<DummyNodes::SaySomething>("SaySomething");
5
6     // 查找文件夹中的所有 XML 文件并注册它们
7     // 我们将使用 std::filesystem::directory_iterator
8     std::string search_directory = "./";
9
10    using std::filesystem::directory_iterator;
11    for (auto const& entry : directory_iterator(search_directory))
12    {
13        if( entry.path().extension() == ".xml")
14        {
15            factory.registerBehaviorTreeFromFile(entry.path().string());
16        }
17    }
18    // 在我们这个特定案例中，相当于：
19    // factory.registerBehaviorTreeFromFile("./main_tree.xml");
20    // factory.registerBehaviorTreeFromFile("./subtree_A.xml");
21    // factory.registerBehaviorTreeFromFile("./subtree_B.xml");
22
23    // 你可以创建 MainTree，子树将被自动添加
24    std::cout << "----- MainTree tick ----" << std::endl;
25    auto main_tree = factory.createTree("MainTree");
26    main_tree.tickWhileRunning();
27
28    // ... 或者你也可以只创建其中一个子树
29    std::cout << "----- SubA tick ----" << std::endl;
30    auto subA_tree = factory.createTree("SubTreeA");
31    subA_tree.tickWhileRunning();
32
33    return 0;
34 }
35 /* 预期输出：
36
37 已注册的行为树：
38 - MainTree
39 - SubTreeA
40 - SubTreeB
41 ----- MainTree tick ----
42 Robot says: starting MainTree
43 Robot says: Executing Sub_A
44 Robot says: Executing Sub_B
45 ----- SubA tick ----
46 Robot says: Executing Sub_A

```

方法二：在XML中使用<include>标签

修改main_tree.xml

代码块

```
1  <root BTCPP_format="4">
2      <include path="./subtree_A.xml" />
3      <include path="./subtree_B.xml" />
4      <BehaviorTree ID="MainTree">
5          <Sequence>
6              <SaySomething message="starting MainTree" />
7              <SubTree ID="SubTreeA" />
8              <SubTree ID="SubTreeB" />
9          </Sequence>
10     </BehaviorTree>
11 </root>
```

- `<include path="...">` 告诉加载器在解析 `main_tree.xml` 时，先去加载指定的文件。
- 路径是相对于当前 XML 文件的。

在像往常一样创建树

代码块

```
1  factory.createTreeFromFile("main_tree.xml")
```

十三、节点参数与默认值配置

1. 向节点传递额外参数

前面的示例我们被“强制”提供一个具有以上签名构造函数

代码块

```
1  MyCustomNode(const std::string& name, const NodeConfig& config);
```

在某些情况，会希望向类的构造函数额外的参数、指针、引用

从理论上来说，这些参数可以使用输入端口传递，但如果满足不以下条件，这样做就是错误的方式

- 黑板/端口适合运行时动态变化的数据，比如传感器读数、目标位置等
- 如果参数在构建树时就已经确定（例如算法参数、硬件接口指针），并且不会在运行时改变，那么用黑板/端口反而会增加不必要的复杂性和开销
- 文档强调：不要滥用黑板，固定参数应该直接通过构造函数或初始化函数传入

方法一：修改构造函数，直接传递参数

原理：

- 在自定义节点的构造函数中，除了必需的name和config，再添加你需要的参数
- 注册节点时，将这些参数一并传给工厂

示例：

代码块

```
1  // Action_A 的构造函数与默认的不同
2  class Action_A: public SyncActionNode
3  {
4
5  public:
6      // 传递给构造函数的额外参数
7      Action_A(const std::string& name, const NodeConfig& config,
8              int arg_int, std::string arg_str):
9          SyncActionNode(name, config),
10         _arg1(arg_int),
11         _arg2(arg_str) {}
12
13     // 此示例不需要任何端口
14     static PortsList providedPorts() { return {}; }
15
16     // tick() 可以访问私有成员
17     NodeStatus tick() override;
18
19 private:
20     int _arg1;
21     std::string _arg2;
22 };
```

注册方式：

代码块

```

1  BT::BehaviorTreeFactory factory;
2  // 将参数 42 和 "hello world" 传递给每个新建的 Action_A 实例
3  factory.registerNodeType<Action_A>("Action_A", 42, "hello world");
4  // 或者显式指定模板参数类型
5  // factory.registerNodeType<Action_A, int, std::string>("Action_A",
    42, "hello world");

```

- 工厂在创建Action_A节点时，会自动调用带着些参数的构造函数
- 注意：所有用“Action_A”标签创建的节点都会获得相同的参数值，如果每个实例需要不同值，看方法二

方法二：使用initialize方法，按需初始化

适用场景：

- 同一个节点类型的不同实例需要不同的参数值（例如多个移动目标点，但每个点的坐标固定且已知）
- 你希望XML中只写标签，参数值在代码中按实例分别设置

原理：

- 节点仍然使用标准构造函数（只有name和config）
- 添加一个自定义的initialize(...)方法，在树创建后、执行前，通过遍历树的方式为每个节点实例设置参数

示例：

代码块

```

1  class Action_B: public SyncActionNode
2  {
3
4  public:
5      // 构造函数看起来和通常一样
6      Action_B(const std::string& name, const NodeConfig& config):
7          SyncActionNode(name, config) {}
8
9      // 我们希望这个方法在第一次 tick() 之前被调用一次
10     void initialize(int arg_int, const std::string& arg_str)
11     {
12         _arg1 = arg_int;
13         _arg2 = arg_str;
14     }
15
16     // 此示例不需要任何端口
17     static PortsList providedPorts() { return {}; }

```

```

18
19     // tick() 可以访问私有成员
20     NodeStatus tick() override;
21
22     private:
23         int _arg1;
24         std::string _arg2;
25     };

```

注册与初始化:

代码块

```

1  BT::BehaviorTreeFactory factory;
2  factory.registerNodeType<Action_B>("Action_B"); // 标准注册
3
4  // 从 XML 创建树 (树中包含多个 <Action_B> 标签)
5  auto tree = factory.createTreeFromText(xml_text);
6
7  // 定义一个访问器, 遍历所有节点, 找到 Action_B 实例并调用 initialize
8  auto visitor = [] (BT::TreeNode* node) {
9      if (auto* action_b = dynamic_cast<Action_B*>(node)) {
10         action_b->initialize(69, "interesting_value"); // 每个实例可传
            不同值
11     }
12 };
13 tree.applyVisitor(visitor); // 应用访问器, 初始化所有 Action_B 节点

```

- applyVisitor会遍历树中的所有节点, 对每个节点调用visitor函数
- 在visitor中, 可以根据节点的名称或其他属性, 给不同的实例传入不同的值
- 这样, XML中的每个<Action_B>标签都可以在代码中被赋予个性化的参数

2. 默认端口值

在大型行为树中, 很多端口经常使用相同的值 (例如固定的坐标、常用的黑板键), 每次在XML专用写一遍会很冗余, 通过为端口设置默认值, 可以让XML更简洁

2.1. 输入端口 (InputPort) 的默认值设置

在节点的providedPorts()中声明输入端口时, 可以附加一个默认值参数, 支持三种方式
比如我们先自定义一个类型Potion2D

代码块

```

1    static PortsList providedPorts()
2    {
3        return {
4            BT::InputPort<Point2D>("input"),
5            BT::InputPort<Point2D>("pointA", Point2D{1, 2}, "默认值为 x=1,
        y=2"),
6            BT::InputPort<Point2D>("pointB", "3,4", "默认值为 x=3,
        y=4"),
7            BT::InputPort<Point2D>("pointC", "{point}", "默认指向黑板条目
        {point}"),
8            BT::InputPort<Point2D>("pointD", "{=}", "默认指向黑板条目
        {pointD}")
9        };
10   }

```

第一个端口 (input) 没有默认值，因此必须在XML中提供值或黑板条目

2.1.1直接提供C++值

代码块

```

1    BT::InputPort<Point2D>("pointA", Point2D{1, 2}, "描述：默认值为
        x=1, y=2")

```

当XML中没有为pointA指定值时，节点将收到默认的Point2D (1,2)

2.1.2提供字符串（利用convertFromString转换）

代码块

```

1    BT::InputPort<Point2D>("pointB", "3,4", "默认值为 x=3, y=4")

```

前提是已经为Point2D特化了convertFromString（例如能从“3,4”解析出坐标）。

最终效果与直接传入Point2D{3,4}相同，但允许用字符串表达默认值，更适合在XML中直接写

2.1.3默认指向某个黑板条目

BT::InputPort<Point2D>("pointC", "{point}", "默认从黑板 point 读取")

如果XML中没有为pointC指定值，节点会默认尝试从黑板的point键读取输入

这是一种“间接默认值”，适用于数据通常来自黑板的情况

2.1.4特殊语法"{=}"

BT::InputPort<Point2D>("pointD", "{=}", "默认从同名的黑板键 pointD 读取")

"{=}"是简写，表示默认使用与端口名相同的黑板键

2.2.输出端口（OutputPort）的默认值

输出带端口的默认值相对受限，因为它们只能指向黑板条目（不能直接给一个固定值）仍然可以用"{=}"简写

代码块

```
1 BT::OutputPort<Point2D>("result", "{target}", "默认写入黑板 target")
```

如果XML中没有位result指定黑板键，则默认写入target键

如果希望默认写入与端口同名的键，可用"{=}"

注意：输出端口不能设置固定的数值默认值，因为输出端口的作用就是向黑板写数据，所以默认值必须是一个黑板键

十四、脚本语言与前后置条件

1.脚本语言简介

1.1.脚本语言作用

引入了一套内置的脚本语言，允许在XML中直接操作黑板变量，而无需编写额外的C++节点，脚本语言主要用于：

- 初始化或修改黑板条目（如<Script code="...">）。
- 在<Precondition>节点中编写条件表达式，动态决定是否执行子树

脚本中的变量名就是黑板键名，读写它们相当于操作黑板

1.2.基本语法与数据类型

1.2.1赋值

代码块

```
1 变量名 := 值      // 如果黑板中不存在该变量，则创建新条目
2 变量名 = 值       // 如果黑板中不存在该变量，则抛出异常
```

- :=用于首次赋值或后续修改，自动创建条目

- =只能用于已存在条目，否则报错

示例：

代码块

```
1  msg := 'hello world'      // 字符串（必须用单引号）
2  A    := 42                 // 整数
3  B    = 3.14                // 浮点数（假设 B 已存在）
4  flag := true               // 布尔值（true/false）
```

1.2.2算数运算符

支持+、-、*、/及其对应的赋值运算符+=、-=、*=、/=

代码块

```
1  A := 7
2  B := 5
3  B *= 2                // B = 10
4  C := (A * 3) + B      // C = 21 + 10 = 31
```

+也可用于字符串拼接

1.2.3位运算符

只能用于整数，否则异常

代码块

```
1  value := 0x7F
2  val_A := value & 0x0F    // 按位与 → 0x0F
3  val_B := value | 0xF0    // 按位或 → 0xFF
4  val_C := ~value          // 按位取反
```

支持|（或）、&（与）、……（异或）、~（取反）

1.2.4逻辑与比较运算符

返回布尔值

代码块

```
1  val_A := true
2  val_B := 5 > 3          // true
3  val_C := (val_A == val_B) // true
```

```
4  val_D := (val_A && val_B) || !val_C  // true
```

比较运算符：== != < <= > >=

逻辑运算符：&& || !

布尔字面量：true / false（注意必须小写，保存到黑板时为 1 和 0）。

1.2.5三元运算符

代码块

```
1  result := (条件) ? 值1 : 值2
```

示例：

代码块

```
1  val_B := (val_A > 1) ? 42 : 24
```

1.2.6字符串拼接

代码块

```
1  greeting := 'Hello, ' + name  // name 是另一个字符串变量
```

1.3.多语句书写

用分号分割多个语句：

代码块

```
1  A := 42; B := 24; msg := 'ok'
```

1.4.在XML中使用脚本

1.4.1Script节点

内置动作节点，用于执行一段脚本

代码块

```
1  <Script code=" msg:='hello world' " />
```

```
2 <Script code=" A:=THE_ANSWER; B:=3.14; color:=RED " />
```

脚本中对黑板的修改会立即生效，后续节点可通过端口读取

1.4.2Precondition节点

用于条件判断，只有条件成立时才执行其唯一子节点，否则返回指定状态

代码块

```
1 <Precondition if="A > B && color != BLUE" else="FAILURE">
2     <Sequence>
3         <!-- 子节点 -->
4     </Sequence>
5 </Precondition>
```

- if后跟一个布尔表达式
- else可选，指定条件不成立时返回的状态（默认为FAILURE，也可设为SUCCESS）

1.5.完整的C++示例（含注册枚举）

代码块

```
1 #include "behaviortree_cpp/bt_factory.h"
2 #include "dummy_nodes.h" // 包含 SaySomething 等
3
4 enum Color { RED = 1, BLUE = 2, GREEN = 3 };
5
6 int main() {
7     BT::BehaviorTreeFactory factory;
8     factory.registerNodeType<DummyNodes::SaySomething>
9         ("SaySomething");
10
11     // 注册枚举类型，脚本中可直接使用 RED、BLUE、GREEN
12     factory.registerScriptingEnums<Color>(); // 注意 magic_enum 默认
13     // 范围 [-128,128]
14
15     // 手动注册一个常量，不受范围限制
16     factory.registerScriptingEnum("THE_ANSWER", 42);
17
18     // XML 文本
19     const char* xml_text = R"(
20         <root BTCPP_format="4">
21             <BehaviorTree>
22                 <Sequence>
```

```

21             <Script code=" msg:='hello world' " />
22             <Script code=" A:=THE_ANSWER; B:=3.14; color:=RED
    " />
23             <Precondition if="A>B && color!=BLUE"
else="FAILURE">
24                 <Sequence>
25                     <SaySomething message="{A}"/>
26                     <SaySomething message="{B}"/>
27                     <SaySomething message="{msg}"/>
28                     <SaySomething message="{color}"/>
29                 </Sequence>
30             </Precondition>
31         </Sequence>
32     </BehaviorTree>
33 </root>
34 );
35
36     auto tree = factory.createTreeFromText(xml_text);
37     tree.tickWhileRunning();
38     return 0;
39 }

```

输出：

代码块

```

1  Robot says: 42.000000
2  Robot says: 3.140000
3  Robot says: hello world
4  Robot says: 1.000000

```

2.前置条件与后置条件

借助之前介绍的脚本语言功能，BT.CPP引入了前置条件和后置条件的概念，即可以在节点的实际tick()之前或之后运行的脚本

所有的节点都支持前置条件和后置条件，并且不需要对C++代码进行任何修改

（脚本的目标不是编写复杂的代码，而是在非常简单的用例中提高对树的可读性并减少对自定义C++节点的需求）

2.1.前置条件

前置条件是在节点开始执行（即从IDLE变为其他状态）之前检查的脚本，支持四种属性，分别用于不同的场景

属性	作用	评估时机
<code>_skipIf</code>	如果条件为真，则跳过该节点（相当于节点未执行，返回SKIPPED）	仅在空闲时（IDLE）状态时（一次）
<code>_failureIf</code>	如果条件为真，则跳过并直接返回FAILURE	仅在空闲时（IDLE）状态时（一次）
<code>_successIf</code>	如果条件为真，则跳过并直接返回SUCCESS	仅在空闲（IDLE）状态时（一次）
<code>_while</code>	条件为假时跳过节点；若节点正在运行且条件变假，则立即中止	空闲和运行中（每次滴答）

重要区别：

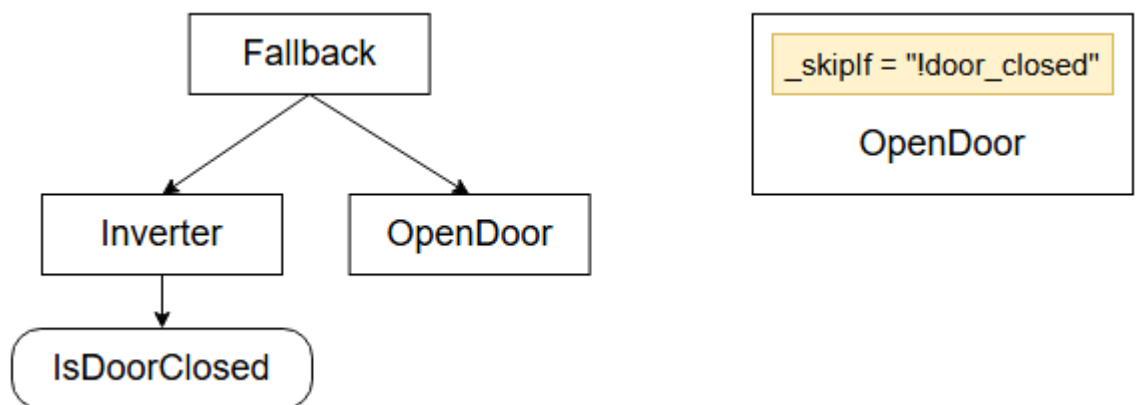
- `_skipIf` / `_failureIf` / `_successIf` 仅在节点第一次从IDLE状态被tick时检查一次，之后即使条件变化也不再重新评估
- `_while`在每次tick时都会检查，包括节点处于RUNNING状态时，如果条件变为假，会立即中止节点（调用onHalted）并返回SKIPPED

评估顺序：

当一个节点同时定义了多个前置条件时，评估顺序为：`_failureIf` -> `_successIf` -> `_skipIf` -> `_while`，一旦某个条件满足，后续条件不再检查

示例：替代Fallback+Inverter

以前如果想实现“如果门关着就开门”，需要用Fallback+Inverter+IsDoorClosed，这样的结构：



代码块

```
1 <Fallback>
```

```
2      <Inverter>
3          <IsDoorClosed/>
4      </Inverter>
5      <OpenDoor/>
6  </Fallback>
```

如果不适用自定义的条件节点IsDoorOpen，而是将布尔值存储在一个名为door_closed的条目中，需要在OpenDoor上加上一个前置条件：

代码块

```
1  <OpenDoor _skipIf="!door_closed"/>
```

使用<Precondition>进行每次tick评估

当你需要在每次tick时（不仅仅是节点启动时）检查一个条件时，需要使用<Precondition>装饰节点，而不是内联属性

这在反应式序列（ReactiveSequence）或反应式回退（ReactiveFallback）中特别有用，因为希望条件在运行中的子节点每次都没tick时都被重新评估

代码块

```
1  <!-- 这会在每次滴答时检查条件 -->
2  <Precondition if="battery_ok" else="RUNNING">
3      <MoveToGoal/>
4  </Precondition>
```

当使用else="RUNNING"时，如果条件在子节点运行时变为假，装饰器会返回运行中（RUNNING）（保持树存活），而不是立即返回失败或已跳过

将此与内联属性进行比较：

代码块

```
1  <!-- 这仅在 MoveToGoal 启动时检查条件 -->
2  <MoveToGoal _successIf="battery_ok"/>
```

内联_successIf仅在MoveToGoal从空闲（IDLE）状态转换时评估一次，如果battery_ok在MoveToGoal运行时发生变化，改变话将被忽略

2.2后置条件（Post Conditions）

后置条件在节点执行完成后（无论返回什么状态）执行脚本，用于根据结果更新黑板或进行其他处理

属性	作用	触发时机
_onSuccess	节点返回SUCCESS时执行	成功时
_onFailure	节点返回FAILURE时执行	失败时
_post	无论成功或失败都执行	成功或失败时
_onHalted	节点被中止（halt）时执行	中止时

示例：替代Fallback+SetBlackboard

以前想根据导航结果设置错误码，需要用一個Fallback嵌套Sequence和ForceFailure：

代码块

```
1  <Fallback>
2    <Sequence>
3      <MoveBase goal="{target}"/>
4      <SetBlackboard output_key="result" value="0" />
5    </Sequence>
6    <ForceFailure>
7      <SetBlackboard output_key="result" value="-1" />
8    </ForceFailure>
9  </Fallback>
```

现在只需要在MoveBase上直接加后置条件

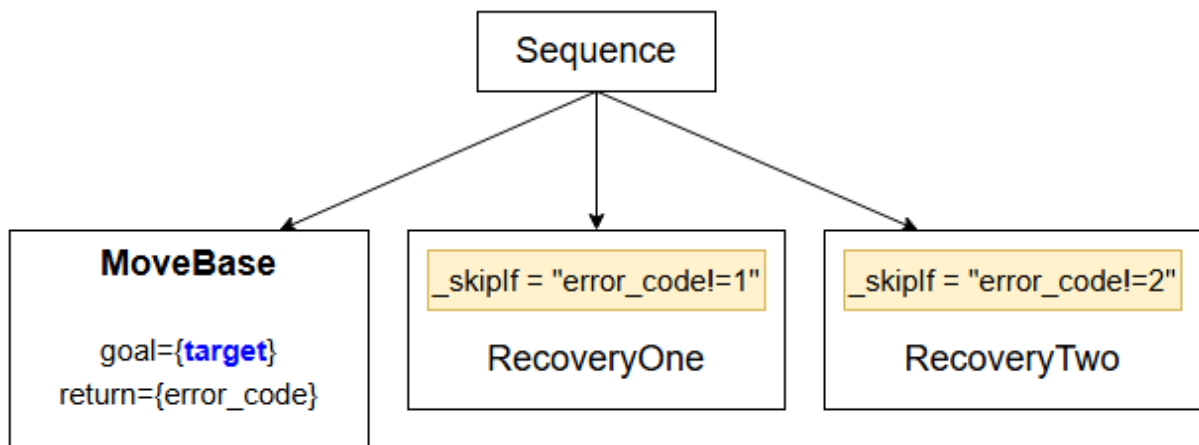
代码块

```
1  <MoveBase goal="{target}"
2    _onSuccess="result:=OK"
3    _onFailure="result:=ERROR"/>
```

2.3.设计模式：错误码

2.3.1根据错误码选择分支

有时动作的失败原因不同，需要走不同的处理路径。可以在动作节点当中设置一个错误码输出端口，然后在后续分支中用前置条件检查该错误码



例如：

代码块

```

1  <Sequence>
2      <MoveBase goal="{target}" error_code="{error}"/>
3      <Fallback>
4          <Condition _skipIf="error != 1"/>    <!-- 错误码为1时的处理 -->
5      >
6          <Condition _skipIf="error != 2"/>    <!-- 错误码为2时的处理 -->
7      >
8  </Fallback>
9  </Sequence>
  
```

这样就不用写复杂的Fallback嵌套

2.3.2状态机风格的节点

可以在节点上加上状态前置/后置条件，使节点的行为依赖于机器人所处的状态，并自动更新状态

例如要给着陆节点：

代码块

```

1  <Lander altitude="{alt}"
2      _skipIf="state != DO_LANDING"    <!-- 只有在 DO_LANDING 状态
    才执行 -->
3      _onSuccess="state:=LANDED"        <!-- 成功后切换到 LANDED -->
4      _onFailure="state:=LANDING_FAILED"/>    <!-- 失败后切换 -->
  
```

这样节点变成声明式：它明确表示“只有特定状态才执行，执行后更新状态”。可以将这个节点自由地移动到树的任何位置，而不用修改内部逻辑

十五、全局黑板与数据架构

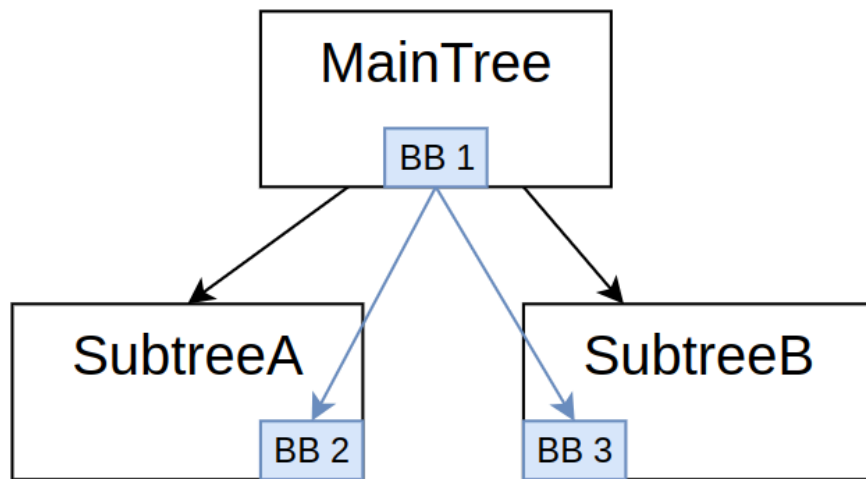
通常情况下，BT.CPP强调“作用域黑板”，即每个子树拥有自己独立的黑板，通过端口重映射来传递数据。这有助于模块化，避免命名冲突

但在某些场景下，需要一块“全局黑板”真正共享全局的数据

- 全局单例对象（如硬件接口、日志器）无法通过端口简单传递
- 机器人的全局状态（如电池电量、当前模式）
- 在主循环（行为树外部）读写的数据，例如传感器输入、用户交互

全局黑板提供了一种便捷的方式，让所有子树都能直接访问同一份数据，而无需每次都写重映射

1.黑板的层次结构



每个子树都有自己的黑板，它们形成父子关系，与树的层级一致，例如：

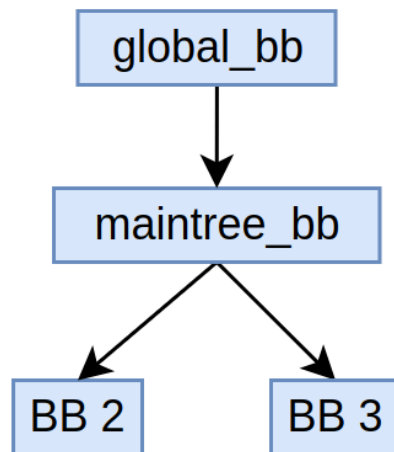
- 主树（MainTree）有自己的黑板BB1
- 主树下的SubTreeA和SubATreeB分别有自己的黑板BB2、BB3，它们的父黑板是BB1

这些独立黑板的生命周期与其各自的子树耦合

可以实现一个外部的“全局黑板”，如下：

代码块

```
1  auto global_bb = BT::Blackboard::create(); //全局黑板（最顶层）
2  auto maintree_bb = BT::Blackboard::create(global_bb); //主树黑板，父为
   global_bb
3  auto tree = factory.createTree("MainTree", maintree_bb);
```



这样，global_bb成为整个黑板层次结构的根，所有子树在查找键时，如果自己及祖先（直到主树）都没有，就会最终找到global_bb。并且global_bb的生命周期独立于树，即使树被销毁，它仍然存在，此外，使用set和get方法也可以轻松访问

2.在XML和脚本中访问全局黑板

为了能够明确指定从顶层黑板（即global_bb）读取或写入，而不是经过中间的父子查找，BT.CPP引入了@前缀

- 在XML端口赋值时：<PrintNumber val="{@value}" />表示端口val将直接绑定到顶层黑板（即我们创建的global_bb）中的value键，而不是本地或父级黑板
- 在脚本语言中也可以使用：<Script code="@value_sqr := @value * @value" />表示读取顶层黑板中的value，计算结果后写入顶层黑板的value_sqr

这样，任何子树，（无论嵌套多深）都可以通过@直接访问全局数据，无需进行任何重映射

3.完整示例

示例中有两个节点：

- PrintNumber：输入端口val，打印该值
- 主树MainTree包含一个PrintNumber（名为main_print）和一个子树MySub
- 子树MySub包含一个PrintNumber（名为sub_print）和一个Script节点

XML：

代码块

```
1    <BehaviorTree ID="MainTree">
2      <Sequence>
3        <PrintNumber name="main_print" val="{@value}" />
4        <SubTree ID="MySub"/>
5      </Sequence>
6    </BehaviorTree>
7
```

```

8     <BehaviorTree ID="MySub">
9         <Sequence>
10             <PrintNumber name="sub_print" val="{@value}" />
11             <Script code="@value_sqr := @value * @value" />
12         </Sequence>
13     </BehaviorTree>

```

- 两个PrintNumber都通过{@value}读取顶层黑板的value键
- Script读取顶层黑板的value，计算平方后写入顶层黑板的value_sqr

C++:

代码块

```

1  class PrintNumber : public BT::SyncActionNode
2  {
3  public:
4      PrintNumber(const std::string& name, const BT::NodeConfig& config)
5          : BT::SyncActionNode(name, config)
6      {}
7
8      static BT::PortsList providedPorts()
9      {
10         return { BT::InputPort<int>("val") };
11     }
12
13     NodeStatus tick() override
14     {
15         const int val = getInput<int>("val").value();
16         std::cout << "[" << name() << "] val: " << val << std::endl;
17         return NodeStatus::SUCCESS;
18     }
19 };
20
21 int main()
22 {
23     BehaviorTreeFactory factory;
24     factory.registerNodeType<PrintNumber>("PrintNumber");
25     factory.registerBehaviorTreeFromText(xml_main);
26
27     // 没有人会拥有此黑板的所有权
28     auto global_bb = BT::Blackboard::create();
29     // "MainTree" 将拥有 maintree_bb
30     auto maintree_bb = BT::Blackboard::create(global_bb);
31     auto tree = factory.createTree("MainTree", maintree_bb);
32

```

```

33 // 我们可以直接与 global_bb 交互
34 for(int i = 1; i <= 3; i++)
35 {
36     // 写入条目 "value"
37     global_bb->set("value", i);
38     // 滴答树
39     tree.tickOnce();
40     // 读取条目 "value_sqr"
41     auto value_sqr = global_bb->get<int>("value_sqr");
42     // 打印
43     std::cout << "[主循环] value: " << i
44                 << " value_sqr: " << value_sqr << "\n\n";
45 }
46 return 0;
47 }

```

执行结果：

代码块

```

1  [main_print] val: 1
2  [sub_print] val: 1
3  [主循环] value: 1 value_sqr: 1
4
5  [main_print] val: 2
6  [sub_print] val: 2
7  [主循环] value: 2 value_sqr: 4
8
9  [main_print] val: 3
10 [sub_print] val: 3
11 [主循环] value: 3 value_sqr: 9

```

执行过程：

- i. 主循环每次将i写入全局黑板value
- ii. tree.tickOnce()执行一次树：
 - main_print通过{@value}读取全局value，打印
 - 进入子树MySub，sub_print同样读取全局value，打印
 - 脚本@value_sqr := @value * @value读取全局value，计算平方后写入全局value_sqr。
- iii. 主循环读取全局value_sqr并打印

输出展示了全局黑板在不同子树和主循环之间的共享效果

十六、模拟测试与节点替换

在真实系统中，某些节点可能依赖硬件（如机器人底盘）、外部服务（如导航）或复杂计算，在测试时难以直接使用，BT.CPP提供了替换规则，可以在创建树之前，指定哪些节点应该被替换成什么。

1. 替换规则的基本用法

每个节点都有一个fullPath，像文件系统中的路径，反映了节点在子树中的位置，例如：

- 如果XML中有<SaySomething name="talk"/>，那么该节点fullPath就是"talk"（因为它在主树顶层，没有前缀）
- 如果有<SubTree ID="MySub" name="mysub"/>内部又有<AlwaysSuccess name="action_subA"/>，那么action_subA的fullPath就是"mysub/action_subA"。

替换规则通过BehaviorTreeFactory的addSubstitutionRule方法添加，规则由两部分组成：

- 匹配模式：一个字符串（可含通配符*），与被替换节点的fullPath进行匹配
- 目标节点类型或配置：指定替换成哪个已注册的节点类型，或者提供一个TestNodeConfig配置对象

例如，给定一下XML

代码块

```
1 <SaySomething name="talk" message="hello world"/>
```

我们可能想用另一个名为TestMessage的节点替换此节点

相应的替换通过一下程序完成

代码块

```
1 factory.addSubstitutionRule("talk", "TestMessage");
```

匹配fullPath为"talk"节点（这里是<SaySomething name="talk"/>），将其替换为已注册的TestSaySomething节点

代码块

```
1 factory.addSubstitutionRule("mysub/action_*", "TestAction");
```

匹配"mysub/"下所有以"action_"开头的节点（如"mysub/action_subA"、"mysub/action_subB"），替换为TestAction节点

2.内置的TestNode

TestNode是一个内置的动作节点，专为测试设计，可以灵活配置：

- 返回状态：可以设置为SUCCESS或FAILURE
- 同步/异步：可以设置为异步，并指定一个延迟时间（async_delay），在此期间返回RUNNING，超时后返回指定状态
- 后置脚本：执行后可以运行一段脚本（post_script），用于模拟输出端口或修改黑板

配置示例：

代码块

```
1  BT::TestNodeConfig test_config;
2  test_config.async_delay = std::chrono::milliseconds(2000); // 异步等待 2
    秒
3  test_config.return_status = BT::NodeStatus::SUCCESS;        // 最终返回成功
4  test_config.post_script = "msg ='message SUBSTITUED'";      // 执行后修改
    黑板
5  factory.addSubstitutionRule("last_action", test_config);
```

将fullPath为“last_action”的节点替换为一个配置好的TestNode，该节点会先返回2秒RUNNING，然后返回SUCCESS，并执行脚本将黑板上的msg改为指定值

3.完整实例

在此示例中，将看到：

- 使用替换规则将一个节点替换为另一个节点
- 如何使用内置的TestNode
- 通配符匹配的示例
- 如何在运行时通过JSON文件传递这些规则

XML：

代码块

```
1  <root BTCPP_format="4">
2    <BehaviorTree ID="MainTree">
3      <Sequence>
4        <SaySomething name="talk" message="hello world"/>
5        <Fallback>
6          <AlwaysFailure name="failing_action"/>
7          <SubTree ID="MySub" name="mysub"/>
8        </Fallback>
9        <SaySomething message="before last_action"/>
```

```

10         <Script code="msg:='after last_action'"/>
11         <AlwaysSuccess name="last_action"/>
12         <SaySomething message="{msg}"/>
13     </Sequence>
14 </BehaviorTree>
15
16 <BehaviorTree ID="MySub">
17     <Sequence>
18         <AlwaysSuccess name="action_subA"/>
19         <AlwaysSuccess name="action_subB"/>
20     </Sequence>
21 </BehaviorTree>
22 </root>

```

C++:

代码块

```

1  int main(int argc, char** argv)
2  {
3      BT::BehaviorTreeFactory factory;
4      factory.registerNodeType<SaySomething>("SaySomething");
5
6      // 我们使用 lambda 和 registerSimpleAction 来创建
7      // 一个我们想要替换给定节点的“虚拟”节点
8
9      // 简单节点，只打印其名称并返回 SUCCESS
10     factory.registerSimpleAction("DummyAction", [](BT::TreeNode& self){
11         std::cout << "DummyAction 正在替换: " << self.name() << std::endl;
12         return BT::NodeStatus::SUCCESS;
13     });
14
15     // 旨在替换 SaySomething 的动作节点。
16     // 它将尝试使用输入端口 "message"
17     factory.registerSimpleAction("TestSaySomething", [](BT::TreeNode& self)
18     {
19         auto msg = self.getInput<std::string>("message");
20         if (!msg)
21         {
22             throw BT::RuntimeError( "缺少必需的输入 [message]: ", msg.error() );
23         }
24         std::cout << "TestSaySomething: " << msg.value() << std::endl;
25         return BT::NodeStatus::SUCCESS;
26     });
27     //-----

```



```

28 // 将 "no_sub" 作为第一个参数传递, 以避免添加规则
29 bool skip_substitution = (argc == 2) && std::string(argv[1]) ==
    "no_sub";
30
31 if(!skip_substitution)
32 {
33     // 我们可以使用 JSON 文件来配置替换规则
34     // 或者手动进行配置
35     bool const USE_JSON = true;
36
37     if(USE_JSON)
38     {
39         factory.loadSubstitutionRuleFromJSON(json_text);
40     }
41     else {
42         // 将匹配此通配符模式的节点替换为 TestAction
43         factory.addSubstitutionRule("mysub/action_*", "TestAction");
44
45         // 将名为 [talk] 的节点替换为 TestSaySomething
46         factory.addSubstitutionRule("talk", "TestSaySomething");
47
48         // 此配置将传递给 TestNode
49         BT::TestNodeConfig test_config;
50         // 将节点转换为异步并等待 2000 毫秒
51         test_config.async_delay = std::chrono::milliseconds(2000);
52         // 完成后执行此后置条件
53         test_config.post_script = "msg ='message SUBSTITUED'";
54
55         // 将名为 [last_action] 的节点替换为 TestNode,
56         // 使用 test_config 配置
57         factory.addSubstitutionRule("last_action", test_config);
58     }
59 }
60
61 factory.registerBehaviorTreeFromText(xml_text);
62
63 // 在树的构建阶段, 替换规则将用于实例化测试节点,
64 // 而不是原始节点。
65 auto tree = factory.createTree("MainTree");
66 tree.tickWhileRunning();
67
68 return 0;
69 }

```

JSON格式

如果不想在代码中写规则，可以准备一个JSON文件，然后再运行时加载，或者USE_JSON==false时。

代码块

```
1  {
2      "TestNodeConfigs": {
3          "MyTest": {
4              "async_delay": 2000,
5              "return_status": "SUCCESS",
6              "post_script": "msg ='message SUBSTITUED'"
7          }
8      },
9
10     "SubstitutionRules": {
11         "mysub/action_*": "TestAction",
12         "talk": "TestSaySomething",
13         "last_action": "MyTest"
14     }
15 }
```

- TestNodeConfigs：定义了一个名为“MyTest”的TestNode配置
- SubstitutionRules：将匹配模式映射到目标。“last_action”映射到配置名“MyTest”，其他映射到节点类型名称

之后在C++中加载：

代码块

```
1  factory.loadSubstitutionRuleFromJSON(json_text); // json_text 是字符串或
    文件路径
```

之后创建树，规则就会生效

十七、日志与状态监控

BT.CPP提供了一种在运行时向树添加日志记录器的方法，通常在树创建之后、开始tick之前进行。

日志记录器是一个类，它有一个回调函数，每当树节点改变其状态时就会被调用；这是所谓的观察者模式的一种非侵入式实现

将被调用的回调函数：

代码块

```
1  virtual void callback(
```

```

2      BT::Duration timestamp, // 状态转换发生的时间
3      const TreeNode& node,   // 改变状态的节点
4      NodeStatus prev_status, // 之前的状态
5      NodeStatus status);     // 新的状态

```

可以继承这个接口实现自己的日志器，记录每次状态的变化

1.TreeObserver：内置的统计日志器

TreeObserver 是一个简单的日志记录器实现，它为树中的每个节点收集以下统计信息

代码块

```

1  struct NodeStatistics
2  {
3      // 最后一个有效结果，SUCCESS 或 FAILURE
4      NodeStatus last_result;
5      // 最后一个状态。可以是任何状态，包括 IDLE 或 SKIPPED
6      NodeStatus current_status;
7      // 状态转换次数，不包括转换到 IDLE 的次数
8      unsigned transitions_count;
9      // 转换到 SUCCESS 的次数
10     unsigned success_count;
11     // 转换到 FAILURE 的次数
12     unsigned failure_count;
13     // 转换到 SKIPPED 的次数
14     unsigned skip_count;
15     // 最后一次转换的时间戳
16     Duration last_timestamp;
17 };

```

对于单元测试非常有用：可以验证某个系数是否按预期执行了特定次数

2.唯一标识一个节点

树中可能有多个同名节点，需要一种方式精确指定要查看那个节点的统计信息。

- `TreeNode::UID()`：一个唯一的数字，对应于树的深度优先遍历顺序
- `TreeNode::fullPath()`：
 - 是一个字符串，类似于文件路径，反映了节点在字数层次结构中的位置
 - 例如：“mysub/sub_nested/action_subB”表示在mysub子树下的sub_nested子树中的action_subB节点
 - 如果XML中没有指定name属性，路径会自动生成如“Sequence::5”这样的形式

3.XML示例

代码块

```
1  <root>
2    <BehaviorTree ID="MainTree">
3      <Sequence>
4        <Fallback>
5          <AlwaysFailure name="failing_action"/>
6          <SubTree ID="SubTreeA" name="mysub"/>
7        </Fallback>
8        <AlwaysSuccess name="last_action"/>
9      </Sequence>
10   </BehaviorTree>
11
12   <BehaviorTree ID="SubTreeA">
13     <Sequence>
14       <AlwaysSuccess name="action_subA"/>
15       <SubTree ID="SubTreeB" name="sub_nested"/>
16       <SubTree ID="SubTreeB" />
17     </Sequence>
18   </BehaviorTree>
19
20   <BehaviorTree ID="SubTreeB">
21     <AlwaysSuccess name="action_subB"/>
22   </BehaviorTree>
23 </root>
```

有些节点具有XML属性“name”，而另一些则没有
相应的UID——fullPath对列表是：

代码块

```
1  1 -> Sequence::1
2  2 -> Fallback::2
3  3 -> failing_action
4  4 -> mysub
5  5 -> mysub/Sequence::5
6  6 -> mysub/action_subA
7  7 -> mysub/sub_nested
8  8 -> mysub/sub_nested/action_subB
9  9 -> mysub/SubTreeB::9
10 10 -> mysub/SubTreeB::9/action_subB
11 11 -> last_action
```

4.C++示例

下列程序将：

- 递归打印树结构
- 将TreeObserver附加到树上
- 答应UID/fullPath对
- 收集名为“last——action”的特定节点的统计信息
- 显示观察者收集的所有统计信息

代码块

```
1  int main()
2  {
3      BT::BehaviorTreeFactory factory;
4
5      factory.registerBehaviorTreeFromText(xml_text);
6      auto tree = factory.createTree("MainTree");
7
8      // 辅助函数，用于打印树
9      BT::printTreeRecursively(tree.rootNode());
10
11     // 观察者的目的是保存关于某个节点返回 SUCCESS 或 FAILURE 次数的统计信息
12     // 这对于创建单元测试以及检查是否按预期发生了一组转换特别有用
13     BT::TreeObserver observer(tree);
14
15     // 打印唯一的 ID 和相应的人类可读路径
16     // 路径也应该是唯一的
17     std::map<uint16_t, std::string> ordered_UID_to_path;
18     for(const auto& [name, uid]: observer.pathToUID()) {
19         ordered_UID_to_path[uid] = name;
20     }
21
22     for(const auto& [uid, name]: ordered_UID_to_path) {
23         std::cout << uid << " -> " << name << std::endl;
24     }
25
26     tree.tickWhileRunning();
27
28     // 你可以使用其完整路径或 UID 访问特定的统计信息
29     const auto& last_action_stats = observer.getStatistics("last_action");
30     assert(last_action_stats.transitions_count > 0);
31
32     std::cout << "-----" << std::endl;
33     // 打印所有统计信息
34     for(const auto& [uid, name]: ordered_UID_to_path) {
```

```

35     const auto& stats = observer.getStatistics(uid);
36
37     std::cout << "[" << name
38               << "]" \tT/S/F: " << stats.transitions_count
39               << "/" << stats.success_count
40               << "/" << stats.failure_count
41               << std::endl;
42 }
43
44 return 0;
45 }

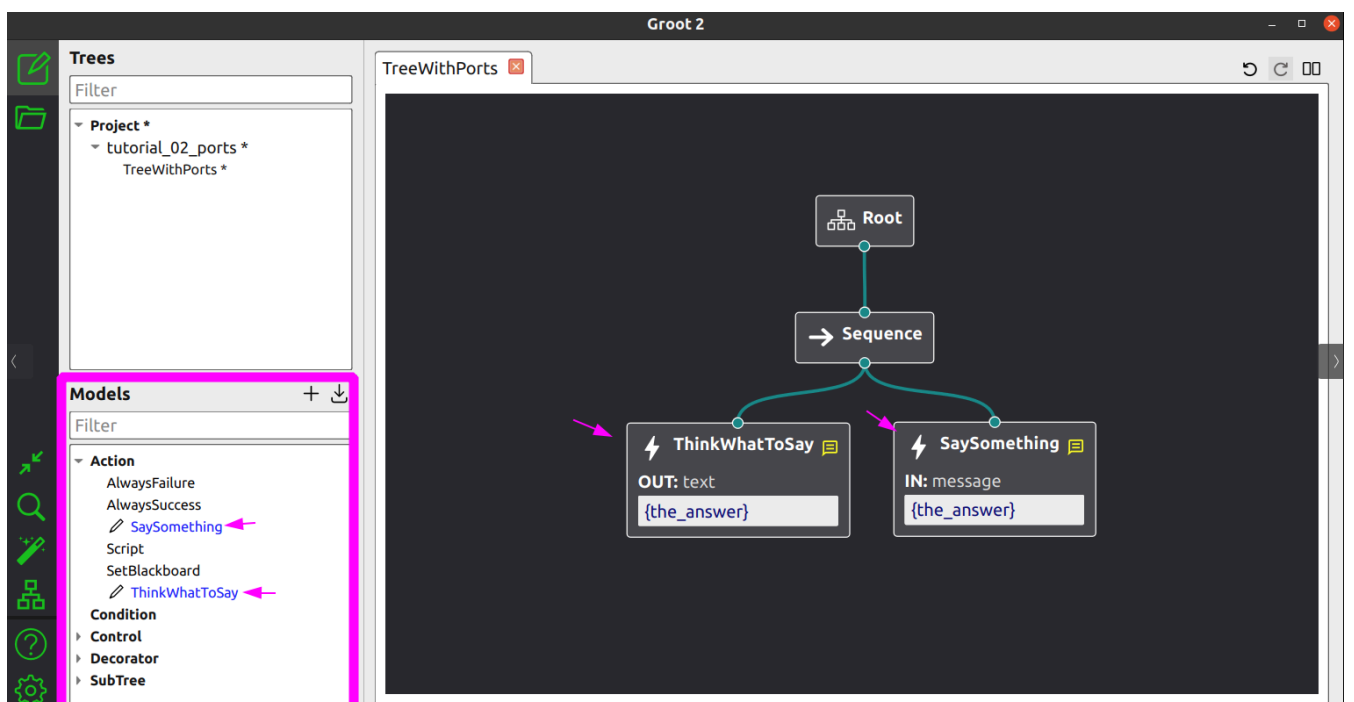
```

十八、Groot2可视化与实时监控

Groot2是用于编辑、监控和与BT.CPP创建的行为树进行交互的官方IDE

1. 树节点模型

Groot2需要一个“树节点模型”



例如，在上图中，Groot需要知道用户定义的节点ThinkWhatToSay和SaySomething存在
此外，还需要：

- 节点类型
- 端口的名称和类型（输入/输出）

这些模型都将用XML表示：

```

1    <TreeNodeModel>
2        <Action ID="SaySomething">
3            <input_port name="message"/>
4        </Action>
5        <Action ID="ThinkWhatToSay">
6            <output_port name="text"/>
7        </Action>
8    </TreeNodeModel>

```

无需手动编写，BT.CPP提供了自动生成函数

代码块

```

1    BT::BehaviorTreeFactory factory;
2    // ... 注册所有自定义节点 ...
3    std::string xml_models = BT::writeTreeNodeModelXML(factory);
4    // 保存 xml_models 到文件，然后在 Groot2 中导入

```

可以将这个XML保存为文件（如models.xml），然后再Groot2中点击“ImportModels”导入；或者直接将模型内容添加到你的.xml或.btproj文件中

2.实时可视化连接

（只有Groot专业版支持）

在C++代码中添加一行

代码块

```

1    BT::Groot2Publisher publisher(tree);

```

这将在BT.CPP执行器和Groot2之间创建一个进程间通信服务，该服务：

- 将整个树结构发送个Groot2，包括上面提到的模型
- 定期更新各个节点的状态（运行中、成功、失败、空闲）
- 发送黑板的值；基本类型如整数、浮点数、字符串等，其他类型需要手动添加
- 允许Groot2插入断电、执行节点替换或故障注入

完整示例：

XML：

代码块

```

1    <root BTCPP_format="4">

```

```

2
3     <BehaviorTree ID="MainTree">
4         <Sequence>
5             <Script code="door_open:=false" />
6             <Fallback>
7                 <Inverter>
8                     <IsDoorClosed/>
9                 </Inverter>
10                <SubTree ID="DoorClosed" _autoremap="true" door_open="
{door_open}"/>
11            </Fallback>
12            <PassThroughDoor/>
13        </Sequence>
14    </BehaviorTree>
15
16    <BehaviorTree ID="DoorClosed">
17        <Fallback name="tryOpen" _onSuccess="door_open:=true">
18            <OpenDoor/>
19            <RetryUntilSuccessful num_attempts="5">
20                <PickLock/>
21            </RetryUntilSuccessful>
22            <SmashDoor/>
23        </Fallback>
24    </BehaviorTree>
25
26 </root>

```

C++:

代码块

```

1  int main()
2  {
3      BT::BehaviorTreeFactory factory;
4
5      // 我们的一组与 CrossDoor 相关的简单节点
6      CrossDoor cross_door;
7      cross_door.registerNodes(factory);
8
9      // Groot2 编辑器需要您注册节点的模型。
10     // 您无需手动编写，可以使用以下命令自动生成。
11     std::string xml_models = BT::writeTreeNodeModelXML(factory);
12
13     factory.registerBehaviorTreeFromText(xml_text);
14     auto tree = factory.createTree("MainTree");
15

```



```

16 // 连接 Groot2Publisher。这将允许 Groot2 获取树并轮询状态更新。
17 BT::Groot2Publisher publisher(tree);
18
19 // 我们希望无限运行此程序
20 while(1)
21 {
22     std::cout << "开始" << std::endl;
23     cross_door.reset();
24     tree.tickWhileRunning();
25     std::this_thread::sleep_for(std::chrono::milliseconds(3000));
26 }
27 return 0;
28 }

```

3. 自定义类型在黑板上的可视化

Groot2通过JSON格式接收黑板中的内容。基本类型已内置支持，但如果你在黑板中存储了自定义类型（如Position2D），则需要为其提供JSON序列化方法

3.1 使用宏BT_JSON_CONVERTERBT_JSON_CONVERTER

给定一个用户定义的类型：

代码块

```

1 struct Position2D
2 {
3     double x;
4     double y;
5 };

```

在文件作用域（任何函数之外）定义转换器：

代码块

```

1 #include "behaviortree_cpp/json_export.h"
2
3 BT_JSON_CONVERTER(Position2D, pos)
4 {
5     add_field("x", &pos.x);
6     add_field("y", &pos.y);
7 }

```

也适用于嵌套类型：

```

1 代码块 struct Waypoint
2  {
3      std::string name;
4      Position2D position;
5      double speed = 1.0;
6  };
7
8  BT_JSON_CONVERTER(Waypoint, wp)
9  {
10     add_field("name", &wp.name);
11     add_field("position", &wp.position);
12     add_field("speed", &wp.speed);
13 }

```

然后在main函数中（创建树之前）注册这些类型：

代码块

```

1  BT::RegisterJsonDefinition<Position2D>();
2  BT::RegisterJsonDefinition<Waypoint>();

```

3.2手动方式

可以自己编写转换函数并注册：

代码块

```

1  void PositionToJson(nlohmann::json& dest, const Position2D& pos) {
2      dest["x"] = pos.x;
3      dest["y"] = pos.y;
4  }
5  // 在 main() 中注册
6  BT::RegisterJsonDefinition<Position2D>(PositionToJson);

```

4.完整示例

代码块

```

1  #include "crossdoor_nodes.h"
2
3  #include "behaviortree_cpp/bt_factory.h"
4  #include "behaviortree_cpp/json_export.h"
5  #include "behaviortree_cpp/loggers/bt_file_logger_v2.h"
6  #include "behaviortree_cpp/loggers/bt_minitrace_logger.h"
7  #include "behaviortree_cpp/loggers/groot2_publisher.h"

```

```

8  #include "behaviortree_cpp/xml_parsing.h"
9
10 #include <cstdlib>
11 #include <string>
12 #include <vector>
13
14 /** 我们使用了教程5中的相同示例，
15  * 但这次还展示了如何连接。
16  */
17
18 // 一个自定义结构体，我们希望在Groot2中可视化
19 struct Position2D
20 {
21     double x;
22     double y;
23 };
24
25 // 这个宏将生成将对象转换为/从JSON转换所需的代码。
26 // 仍然需要在main()中调用 BT::RegisterJsonDefinition<Position2D>()
27 // NOLINTNEXTLINE(misc-use-internal-linkage)
28 BT_JSON_CONVERTER(Position2D, pos)
29 {
30     add_field("x", &pos.x);
31     add_field("y", &pos.y);
32 }
33
34 struct Waypoint
35 {
36     std::string name;
37     Position2D position;
38     double speed = 1.0;
39 };
40
41 // NOLINTNEXTLINE(misc-use-internal-linkage)
42 BT_JSON_CONVERTER(Waypoint, wp)
43 {
44     add_field("name", &wp.name);
45     add_field("position", &wp.position);
46     add_field("speed", &wp.speed);
47 }
48
49 // 简单的动作，更新黑板中的 Position2D 实例，
50 // 并输出向量以复现 Groot2 issue #55/#77
51 // 自定义同步动作节点UpdatePosition
52 // 每次tick都会更新黑板中的Position2值（增加偏移）并生成随机变化的
53 // std::vector<double>
54 // std::vector<std::string>

```

```

55 //std::vector<Waypoint> (动态改变数量, 演示复杂类型的可视化)
56 class UpdatePosition : public BT::SyncActionNode
57 {
58 public:
59     UpdatePosition(const std::string& name, const BT::NodeConfig& config)
60         : BT::SyncActionNode(name, config)
61     {}
62
63     BT::NodeStatus tick() override
64     {
65         _pos.x += 0.2;
66         _pos.y += 0.1;
67         setOutput("pos", _pos);
68
69         // 触发 Groot2 issue #55/#77 的向量 (查看黑板时崩溃)
70         std::vector<double> doubles = { 1.1, 2.2, 3.3, 4.4, 5.5 };
71         setOutput("vec_double", doubles);
72
73         std::vector<std::string> strings = { "hello", "world", "test" };
74         setOutput("vec_string", strings);
75
76         // 生成随机偏移量的辅助函数 [-range, +range]
77         auto randOffset = [](double range) {
78             // NOLINTNEXTLINE(cert-msc30-c,cert-msc50-cpp,concurrency-mt-unsafe)
79             return (static_cast<double>(std::rand())) / RAND_MAX - 0.5) * 2.0 *
range;
80         };
81
82         // 自定义类型的向量 - 在2个和3个航点之间交替
83         std::vector<Waypoint> waypoints;
84         if(_execution_count % 2 == 0)
85         {
86             // 偶数次执行: 2个航点
87             waypoints = {
88                 { "start", { randOffset(5.0), randOffset(5.0) }, 1.0 +
randOffset(0.3) },
89                 { "goal",
90                     { 100.0 + randOffset(10.0), 50.0 + randOffset(10.0) },
91                     0.5 + randOffset(0.2) },
92             };
93         }
94         else
95         {
96             // 奇数次执行: 3个航点
97             waypoints = {

```

```

98         { "start", { randOffset(5.0), randOffset(5.0) }, 1.0 +
randOffset(0.3) },
99         { "checkpoint",
100         { 50.0 + randOffset(15.0), 25.0 + randOffset(15.0) },
101         2.0 + randOffset(0.5) },
102         { "goal",
103         { 100.0 + randOffset(10.0), 50.0 + randOffset(10.0) },
104         0.5 + randOffset(0.2) },
105     };
106 }
107 setOutput("waypoints", waypoints);
108 _execution_count++;
109
110 return BT::NodeStatus::SUCCESS;
111 }
112
113 static BT::PortsList providedPorts()
114 {
115     return {
116         BT::OutputPort<Position2D>("pos"),
117         BT::OutputPort<std::vector<double>>("vec_double", "double类型向量"),
118         BT::OutputPort<std::vector<std::string>>("vec_string", "字符串向
量"),
119         BT::OutputPort<std::vector<Waypoint>>("waypoints", "航点向量"),
120     };
121 }
122
123 private:
124     Position2D _pos = { 0, 0 };
125     int _execution_count = 0;
126 };
127
128 // clang-format off
129 namespace
130 {
131     const char* xml_text = R"(
132     <root BTCPP_format="4">
133
134         <BehaviorTree ID="MainTree">
135             <Sequence>
136                 <Script code="door_open:=false" />
137                 <UpdatePosition pos="{pos_2D}" vec_double="{doubles}" vec_string="{
strings}" waypoints="{waypoints}"/>
138             <Fallback>
139                 <Inverter>
140                     <IsDoorClosed/>
141                 </Inverter>

```

```

142         <SubTree ID="DoorClosed" _autoremap="true" door_open="
{door_open}"/>
143     </Fallback>
144     <PassThroughDoor/>
145 </Sequence>
146 </BehaviorTree>
147
148 <BehaviorTree ID="DoorClosed">
149     <Fallback name="tryOpen" _onSuccess="door_open:=true">
150         <OpenDoor/>
151         <RetryUntilSuccessful num_attempts="5">
152             <PickLock/>
153         </RetryUntilSuccessful>
154         <SmashDoor/>
155     </Fallback>
156 </BehaviorTree>
157
158 </root>
159 )";
160 } // namespace
161 // clang-format on
162
163 int main()
164 {
165     BT::BehaviorTreeFactory factory;
166
167     // 照常注册节点
168     CrossDoor cross_door;
169     cross_door.registerNodes(factory);
170     factory.registerNodeType<UpdatePosition>("UpdatePosition");
171
172     // Groot2 编辑器需要您注册节点的模型。
173     // 您无需手动编写，可以使用以下命令自动生成。
174     [[maybe_unused]] const std::string xml_models =
BT::writeTreeNodesModelXML(factory);
175
176     factory.registerBehaviorTreeFromText(xml_text);
177
178     // 添加此行以允许 Groot2 可视化您的自定义类型
179     BT::RegisterJsonDefinition<Position2D>();
180     BT::RegisterJsonDefinition<Waypoint>();
181
182     auto tree = factory.createTree("MainTree");
183
184     std::cout << "----- XML 文件 -----\n"
185               << BT::WriteTreeToXML(tree, false, false)
186               << "-----\n";

```

```

187
188 // 连接 Groot2Publisher。这将允许 Groot2 获取树并轮询状态更新。
189 const unsigned port = 1667;
190 BT::Groot2Publisher publisher(tree, port);
191
192 // 添加另外两个日志记录器，将状态转换保存到文件。
193 // 两种格式都与 Groot2 兼容
194
195 // 使用轻量级序列化的日志记录
196 BT::FileLogger2 logger2(tree, "t11_groot_howto.btlog");
197 BT::MinitraceLogger minilog(tree, "minitrace.json");
198
199 while(true)
200 {
201     std::cout << "开始" << std::endl;
202     cross_door.reset();
203     tree.tickWhileRunning();
204     std::this_thread::sleep_for(std::chrono::milliseconds(2000));
205 }
206
207 return 0;
208 }

```

这个示例展示了如何将行为树与Groot2深度集成，包括：

- 自定义类型的可视化
- 实时监控节点状态和黑板数据
- 录制日志用于后续回放或分析
- 通过Groot2Publisher实现调试交互

十九、附录：xml模式参考

我们可以看到一棵简单的用xml编写的树

代码块

```

1 <root BTCPP_format="4">
2   <BehaviorTree ID="MainTree">
3     <Sequence name="root_sequence">
4       <SaySomething name="action_hello" message="Hello"/>
5       <OpenGripper name="open_gripper"/>
6       <ApproachObject name="approach_object"/>
7       <CloseGripper name="close_gripper"/>
8     </Sequence>
9   </BehaviorTree>

```

- 在这棵树中，我们首先需要的是<root>根节点，他的下面可以包含一个或多个叶节点。
- 在下面的叶节点中，标签<BehaviorTree>中，具有属性ID
- 在root节点中，要添加属性[BTCPP_format]
- 每个树节点都应该设置一个标签，标签可设置属性name
 - 标签的名称是用于工厂中注册该树节点的ID
 - 属性[name]指的是实例名称，且为可选项
 - 需要添加端口进行配置时，需要输入端口message

1.端口重映射与指向黑板条目的指针

输入/输出端口可以使用黑板中的条目名称（即黑板键/值对的键）进行重映射

黑板键使用以下语法表示：{key_name}

在下面的示例中：

- 序列的第一个子节点打印 “hello”
- 第二个子节点读取并写入黑板中名为” my_message”的条目所包含的值

代码块

```

1  <root BTCPP_format="4" >
2      <BehaviorTree ID="MainTree">
3          <Sequence name="root_sequence">
4              <SaySomething message="Hello"/>
5              <SaySomething message="{my_message}"/>
6          </Sequence>
7      </BehaviorTree>
8  </root>

```

2.紧凑表示法与显示表示法

以下两种语法都是有效的：

代码块

```

1  <SaySomething                name="action_hello" message="Hello World"/>
2  <Action ID="SaySomething"    name="action_hello" message="Hello World"/>

```


将前者称为“紧凑”语法，后者称为“显式”语法。第一个示例如果用显式语法表示，将变成：

代码块

```
1  <root BTCPP_format="4" >
2      <BehaviorTree ID="MainTree">
3          <Sequence name="root_sequence">
4              <Action ID="SaySomething"    name="action_hello"
message="Hello"/>
5              <Action ID="OpenGripper"    name="open_gripper"/>
6              <Action ID="ApproachObject" name="approach_object"/>
7              <Action ID="CloseGripper"   name="close_gripper"/>
8          </Sequence>
9      </BehaviorTree>
10 </root>
```

即使紧凑语法更方便，更易编写，但提供的关于树节点模型的信息太少，像Groot这样的工具需要显式语法或额外信息。这些信息可以使用<TreeNodeModel>标签添加

为了使树的紧凑版本与Groot兼容，XML修改如下：

代码块

```
1  <root BTCPP_format="4" >
2      <BehaviorTree ID="MainTree">
3          <Sequence name="root_sequence">
4              <SaySomething    name="action_hello" message="Hello"/>
5              <OpenGripper    name="open_gripper"/>
6              <ApproachObject name="approach_object"/>
7              <CloseGripper   name="close_gripper"/>
8          </Sequence>
9      </BehaviorTree>
10
11      <!-- 行为树执行器不需要这个，但 Groot 需要 -->
12      <TreeNodeModel>
13          <Action ID="SaySomething">
14              <input_port name="message" type="std::string" />
15          </Action>
16          <Action ID="OpenGripper"/>
17          <Action ID="ApproachObject"/>
18          <Action ID="CloseGripper"/>
19      </TreeNodeModel>
20 </root>
```

3.子树

可以将子树包含在另一个树中，避免在多个位置“复制粘贴”相同的树，降低复杂性

假设想将一些动作封装到名为“GraspObject”的行为树中（省略可选属性[name]）：

代码块

```
1   <root BTCPP_format="4" >
2
3       <BehaviorTree ID="MainTree">
4           <Sequence>
5               <Action ID="SaySomething" message="Hello World"/>
6               <SubTree ID="GraspObject"/>
7           </Sequence>
8       </BehaviorTree>
9
10      <BehaviorTree ID="GraspObject">
11          <Sequence>
12              <Action ID="OpenGripper"/>
13              <Action ID="ApproachObject"/>
14              <Action ID="CloseGripper"/>
15          </Sequence>
16      </BehaviorTree>
17  </root>
```

整个树GraspObject会在SaySomething之后执行

4.包含外部文件

可以包含外部文件，类似于C++中的 `#include <file>` 。可以使用以下标签来实现：

代码块

```
1   <include path="relative_or_absolute_path_to_file">
```

使用这个示例，可以将行为树分割到两个文件中：

代码块

```
1   <!-- 文件 maintree.xml -->
2
3   <root BTCPP_format="4" >
4
5       <include path="grasp.xml"/>
6
```

```

7      <BehaviorTree ID="MainTree">
8          <Sequence>
9              <Action ID="SaySomething" message="Hello World"/>
10             <SubTree ID="GraspObject"/>
11          </Sequence>
12      </BehaviorTree>
13  </root>

```

代码块

```

1  <!-- 文件 grasp.xml -->
2
3  <root BTCPP_format="4" >
4      <BehaviorTree ID="GraspObject">
5          <Sequence>
6              <Action ID="OpenGripper"/>
7              <Action ID="ApproachObject">
8              <Action ID="CloseGripper"/>
9          </Sequence>
10     </BehaviorTree>
11 </root>

```

如果想在ROS功能包内查找该文件，可以使用以下语法：

代码块

```

1  <include ros_pkg="name_package" path="path_relative_to_pkg/grasp.xml"/>

```

二十、BT.CPP与ROS2的集成

BT.CPP提供了一套即用型包装器，可用于快速实现与ROS2交互的树节点：BehaviorTree.ROS2

在系统架构方面，应该：

- 应该有一个集中的“协调器” ROS节点，负责行为的执行。这将在下文称为“任务规划器”，并将使用BT.CPP实现
- 系统的所有其他元素都应是“面向服务”组件，并应将任何业务逻辑和决策制定委托给任务规划器

此外，有些术语需要进行区分

- ROS2节点（rclcpp::Node）：ROS2中通信基本单元
- BT节点（BT::TreeNode）：行为树中的节点

- ROS Action (rclcpp_action) : ROS2中用于长时间运行任务的通信机制
- BT Action (BT::ActionNodeBase) : 行为树中执行具体动作的节点

1.异步ROS Action封装

与ROS交互的推荐方式是通过rclcpp_action

因为:

- 它们的API是异步的, 即用户无需担心创建单独的线程
- 它们可以被中止, 这是实现 `TreeNode::halt()` 和构建反应式行为所需的功能

创建一个调用ROS动作的BT动作:

代码块

```
1  #include <behaviortree_ros2/bt_action_node.hpp>
2  #include "action_tutorials_interfaces/action/fibonacci.hpp"
3
4  // 为简洁起见, 我们定义这些类型
5  using Fibonacci = action_tutorials_interfaces::action::Fibonacci;
6  using GoalHandleFibonacci = rclcpp_action::ServerGoalHandle<Fibonacci>;
7
8  using namespace BT;
9
10 class FibonacciAction: public RosActionNode<Fibonacci>
11 {
12 public:
13     FibonacciAction(const std::string& name,
14                     const NodeConfig& conf,
15                     const RosNodeParams& params)
16         : RosActionNode<Fibonacci>(name, conf, params)
17     {}
18
19     // 此衍生类的特定端口
20     // 应使用 RosActionNode::providedBasicPorts() 与基类的端口合并
21     static PortsList providedPorts()
22     {
23         return providedBasicPorts({InputPort<unsigned>("order")});
24     }
25
26     // 当 TreeNode 被滴答时调用此方法, 它应
27     // 向动作服务器发送请求
28     bool setGoal(RosActionNode::Goal& goal) override
29     {
30         // 从输入端口获取 "order"
31         getInput("order", goal.order);
32         // 如果成功设置目标, 则返回 true
```

```

33     return true;
34 }
35
36 // 收到回复时执行的回调。
37 // 根据回复，你可以决定返回 SUCCESS 或 FAILURE。
38 NodeStatus onResultReceived(const WrappedResult& wr) override
39 {
40     std::stringstream ss;
41     ss << "收到结果: ";
42     for (auto number : wr.result->sequence) {
43         ss << number << " ";
44     }
45     RCLCPP_INFO(logger(), ss.str().c_str());
46     return NodeStatus::SUCCESS;
47 }
48
49 // 当客户端和服务端之间的通信出现错误时调用的回调。
50 // 这将根据返回值将 TreeNode 的状态设置为 SUCCESS 或 FAILURE。
51 // 如果不重写，默认返回 FAILURE。
52 virtual NodeStatus onFailure(ActionNodeErrorCode error) override
53 {
54     RCLCPP_ERROR(logger(), "错误: %d", error);
55     return NodeStatus::FAILURE;
56 }
57
58 // 我们也支持反馈的回调，如
59 // 原始教程中所示。
60 // 通常，此回调应返回 RUNNING，但你可以
61 // 根据反馈的值决定中止
62 // 动作，并认为 TreeNode 已完成。
63 // 在这种情况下，返回 SUCCESS 或 FAILURE。
64 // 取消请求将自动发送到服务器。
65 NodeStatus onFeedback(const std::shared_ptr<const Feedback> feedback)
66 {
67     std::stringstream ss;
68     ss << "收到序列中的下一个数字: ";
69     for (auto number : feedback->partial_sequence) {
70         ss << number << " ";
71     }
72     RCLCPP_INFO(logger(), ss.str().c_str());
73     return NodeStatus::RUNNING;
74 }
75 };

```

步骤：

- 包含头文件 `<behaviortree_ros2/bt_action_node.hpp>`
- 继承 `RosActionNode<ActionType>`
- 重写关键方法：
 - `setGoal()`：填充目标请求，返回true表示成功
 - `onResultReceived()`：处理最终结果，返回SUCCESS或FAILURE
 - `onFeedback()`（可选）：处理反馈，通常返回RUNNING；若需中止，可返回SUCCESS/FAILURE
 - `onFailure()`（可选）：处理通信错误，返回状态
- 在`providedPorts()`中，除了自定义端口，还要调用`providedBasicPorts()`合并基类端口（如服务名称）
- 注册节点时，需要传递`RosNodeParams`结构，包含ROS节点句柄和服务名称

动作客户端的BT版本比原始版本更简单，因为大部分样板代码都封装在`BT::RosAcitonNode`包装器中

注册此节点，需要使用`BT::RosNodeParams`传递`rclcpp::Node`和其他参数

代码块

```
1 // 在 main() 函数中
2 BehaviorTreeFactory factory;
3
4 auto node = std::make_shared<rclcpp::Node>("fibonacci_action_client");
5 // 提供 ROS 节点和动作服务的名称
6 RosNodeParams params;
7 params.nh = node;
8 params.default_port_value = "fibonacci";
9 factory.registerNodeType<FibonacciAction>("Fibonacci", params);
```

2.使用rclcpp::Client（服务）的异步BT::Action

对于ROS服务客户端，也有一个类似的包装器可用，将使用异步接口

下面的示例基于官方教程：

代码块

```
1 #include <behaviortree_ros2/bt_service_node.hpp>
2 #include "example_interfaces/srv/add_two_ints.hpp"
3
4 using AddTwoInts = example_interfaces::srv::AddTwoInts;
5 using namespace BT;
6
```

```

7
8 class AddTwoIntsNode: public RosServiceNode<AddTwoInts>
9 {
10     public:
11
12     AddTwoIntsNode(const std::string& name,
13                     const NodeConfig& conf,
14                     const RosNodeParams& params)
15         : RosServiceNode<AddTwoInts>(name, conf, params)
16     {}
17
18     // 此衍生类的特定端口
19     // 应使用 RosServiceNode::providedBasicPorts() 与基类的端口合并
20     static PortsList providedPorts()
21     {
22         return providedBasicPorts({
23             InputPort<unsigned>("A"),
24             InputPort<unsigned>("B")});
25     }
26
27     // 当 TreeNode 被滴答时调用此方法，它应
28     // 向服务提供者发送请求
29     bool setRequest(Request::SharedPtr& request) override
30     {
31         // 使用输入端口设置 A 和 B
32         getInput("A", request->a);
33         getInput("B", request->b);
34         // 如果准备好发送请求，必须返回 true
35         return true;
36     }
37
38     // 收到应答时调用的回调。
39     // 必须返回 SUCCESS 或 FAILURE
40     NodeStatus onResponseReceived(const Response::SharedPtr& response)
41     override
42     {
43         RCLCPP_INFO(logger(), "和: %ld", response->sum);
44         return NodeStatus::SUCCESS;
45     }
46
47     // 当客户端和服务端之间的通信出现错误时调用的回调。
48     // 这将根据返回值将 TreeNode 的状态设置为 SUCCESS 或 FAILURE。
49     // 如果不重写，默认返回 FAILURE。
50     virtual NodeStatus onFailure(ServiceNodeErrorCode error) override
51     {
52         RCLCPP_ERROR(logger(), "错误: %d", error);
53         return NodeStatus::FAILURE;
54     }
55 }

```

```
53     }  
54   };
```

结尾：

至此笔记结束，参考了官方文档<https://www.behaviortree.dev/docs/intro>，使用了ai进行英文翻译以及部分总结。